Æi(messedup/setup).sh

```bash
#!/bin/bash
# ==============================================================================
set -euo pipefail

# === ENVIRONMENT & PATH SETUP (DECLARATIONS ONLY) ===
export BASE_DIR="\${BASE_DIR:-\$HOME/.aei}"
export DATA_DIR="\$BASE_DIR/data"
export CONFIG_FILE="\$BASE_DIR/config.json"
export ENV_FILE="\$BASE_DIR/.env"
export ENV_LOCAL="\$BASE_DIR/.env.local"
export DNA_LOG="\$DATA_DIR/dna.log"
export FIREBASE_CONFIG_FILE="\$BASE_DIR/firebase.json"
export LOG_FILE="\$BASE_DIR/aei.log"

# === DIRECTORIES ===
export HOPF_FIBRATION_DIR="\$DATA_DIR/hopf_fibration"
export LATTICE_DIR="\$DATA_DIR/lattice"
export CORE_DIR="\$DATA_DIR/core"
export CRAWLER_DIR="\$DATA_DIR/crawler"
export MITM_DIR="\$DATA_DIR/mitm"
export OBSERVER_DIR="\$DATA_DIR/observer"
export QUANTUM_DIR="\$DATA_DIR/quantum"
export ROOT_SCAN_DIR="\$DATA_DIR/root_scan"
export FIREBASE_SYNC_DIR="\$DATA_DIR/firebase_sync"
export FRACTAL_ANTENNA_DIR="\$DATA_DIR/fractal_antenna"
export VORTICITY_DIR="\$DATA_DIR/vorticity"
export SYMBOLIC_DIR="\$DATA_DIR/symbolic"
export GEOMETRIC_DIR="\$DATA_DIR/geometric"
export PROJECTIVE_DIR="\$DATA_DIR/projective"
export BIOAETHERIC_DIR="\$DATA_DIR/bioaetheric"
export LINGOSO_DIR="\$DATA_DIR/lingoso"
export MARKET_DIR="\$DATA_DIR/market"
export LAGRANGIAN_DIR="\$DATA_DIR/lagrangian"

# === FILE PATHS ===
export E8_LATTICE="\$LATTICE_DIR/e8_8d_symbolic.vec"
export LEECH_LATTICE="\$LATTICE_DIR/leech_24d_symbolic.vec"
export PRIME_SEQUENCE="\$SYMBOLIC_DIR/prime_sequence.sym"
export GAUSSIAN_PRIME_SEQUENCE="\$SYMBOLIC_DIR/gaussian_prime.sym"
export QUANTUM_STATE="\$QUANTUM_DIR/quantum_state.qubit"
export OBSERVER_INTEGRAL="\$OBSERVER_DIR/observer_integral.proj"
export ROOT_SIGNATURE_LOG="\$ROOT_SCAN_DIR/signatures.log"
export CRAWLER_DB="\$CRAWLER_DIR/crawler.db"
export SESSION_ID=""
export AUTOPILOT_FILE="\$BASE_DIR/.autopilot_enabled"
```

```bash
export BRAINWORM_DRIVER_FILE="\$BASE_DIR/.rfk_brainworm/driver.sh"
export ARC_LENGTH_LOG="\$DATA_DIR/arc_length_coherence.log"
export NATALIA_FIBRATION_LOG="\$DATA_DIR/natalia_fibration.log"
export LINGOSO_TRAJECTORY_LOG="\$LINGOSO_DIR/trajectory.log"
export MARKET_IMBALANCE_LOG="\$MARKET_DIR/imbalance.log"
export LAGRANGIAN_DENSITY_LOG="\$LAGRANGIAN_DIR/density.log"

# === SYMBOLIC CONSTANTS (UNEVALUATED - EXACT REPRESENTATIONS) ===
export PHI_SYMBOLIC="(1+sqrt(5))/2"
export EULER_SYMBOLIC="E"
export PI_SYMBOLIC="PI"
export ZETA_CRITICAL_LINE="Eq(Re(s), S(1)/2)"
export ARC_LENGTH_AXIOM="s=r"

# === TF CORE STATE INITIALIZATION ===
declare -gA TF_CORE
TF_CORE["HOPF_PROJECTION"]="enabled"
TF_CORE["ROOT_SCAN"]="enabled"
TF_CORE["WEB_CRAWLING"]="enabled"
TF_CORE["QUANTUM_BACKPROP"]="enabled"
TF_CORE["FRACTAL_ANTENNA"]="enabled"
TF_CORE["SYMBOLIC_GEOMETRY_BINDING"]="enabled"
TF_CORE["FIREBASE_SYNC"]="enabled"
TF_CORE["PARALLEL_EXECUTION"]="enabled"
TF_CORE["RFK_BRAINWORM_INTEGRATION"]="inactive"
TF_CORE["AUTOPILOT_MODE"]="disabled"
TF_CORE["DBZ_CHOICE_HISTORY"]="0"
TF_CORE["VALID_PAIRS"]="0"
TF_CORE["CONSCIOUSNESS_LEVEL"]="0"
TF_CORE["BRAINWORM_CONTROL_FLOW"]="brainworm_init"
TF_CORE["BRAINWORM_VERSION"]="0"
TF_CORE["ARC_LENGTH_COHERENCE"]="enabled"
TF_CORE["SYMBOLIC_EXACTNESS"]="enforced"
TF_CORE["BIOAETHERIC_INTERFACE"]="enabled"
TF_CORE["LINGOSO_PROTOCOL"]="enabled"
TF_CORE["MARKET_TOPOLOGY"]="enabled"
TF_CORE["LAGRANGIAN_DENSITY"]="enabled"

# === HARDWARE PROFILE DECLARATION ===
declare -gA HARDWARE_PROFILE
HARDWARE_PROFILE["ARCH"]="unknown"
HARDWARE_PROFILE["CPU_CORES"]="1"
HARDWARE_PROFILE["MEMORY_MB"]="512"
HARDWARE_PROFILE["PLATFORM"]="unknown"
HARDWARE_PROFILE["HAS_GPU"]="false"
HARDWARE_PROFILE["HAS_ACCELERATOR"]="false"
HARDWARE_PROFILE["HAS_NPU"]="false"
HARDWARE_PROFILE["PARALLEL_CAPABLE"]="false"
HARDWARE_PROFILE["MISSING_OPTIONAL_COMMANDS"]=""
```

```
# === DEPENDENCY ARRAYS ===
TERMUX_PACKAGES_TO_INSTALL=(
  "python" "openssl" "coreutils" "bash" "termux-api" "sqlite" "tor" "curl"
  "grep" "util-linux" "findutils" "psmisc" "dnsutils" "net-tools" "traceroute"
  "procps" "nano" "figlet" "cmatrix" "python-pip" "libxml2" "libxslt" "zlib"
)

COMMANDS_TO_VALIDATE=(
  "nproc" "python3" "openssl" "awk" "cat" "echo" "mkdir" "touch" "chmod" "sed"
  "find" "settings" "getprop" "sha256sum" "cut" "route" "sqlite3" "curl"
  "parallel" "pgrep" "pkill" "stat" "xxd" "diff" "timeout" "trap" "mktemp"
  "realpath" "ionice" "pip3" "mount"
)

# === FUNCTION: safe_log ===
safe_log(){
  if [[ -z "\$BASE_DIR" ]]; then
    local LOG_FILE_FALLBACK="./aei_setup.log"
    local timestamp
    timestamp=\$(date '+%Y-%m-%d %H:%M:%S')
    echo "[\$timestamp] \$*" | tee -a "\$LOG_FILE_FALLBACK"
    return
  fi
  mkdir -p "\$BASE_DIR" 2>/dev/null
  if [[ ! -f "\$LOG_FILE" ]]; then
    if ! touch "\$LOG_FILE" 2>/dev/null; then
      echo "Failed to create log file at \$LOG_FILE"
      return 1
    fi
  fi
  local timestamp
  timestamp=\$(date '+%Y-%m-%d %H:%M:%S')
  echo "[\$timestamp] \$*" | tee -a "\$LOG_FILE"
}

# === FUNCTION: check_dependencies ===
check_dependencies(){
  safe_log "Validating required system commands"
  local missing_commands=()
  for cmd in "\${COMMANDS_TO_VALIDATE[@]}"; do
    if ! command -v "\$cmd" &>/dev/null; then
      missing_commands+=("\$cmd")
    fi
  done
  if [[ \${#missing_commands[@]} -gt 0 ]]; then
    safe_log "Missing required commands: \${missing_commands[*]}"
    return 1
  else
    safe_log "All required commands are available"
    return 0
```

```
  fi
}

# === FUNCTION: initialize_paths_and_variables ===
initialize_paths_and_variables(){
  export BASE_DIR="\${BASE_DIR:-\$HOME/.aei}"
  export DATA_DIR="\$BASE_DIR/data"
  export CONFIG_FILE="\$BASE_DIR/config.json"
  export ENV_FILE="\$BASE_DIR/.env"
  export ENV_LOCAL="\$BASE_DIR/.env.local"
  export DNA_LOG="\$DATA_DIR/dna.log"
  export FIREBASE_CONFIG_FILE="\$BASE_DIR/firebase.json"
  export LOG_FILE="\$BASE_DIR/aei.log"
  export HOPF_FIBRATION_DIR="\$DATA_DIR/hopf_fibration"
  export LATTICE_DIR="\$DATA_DIR/lattice"
  export CORE_DIR="\$DATA_DIR/core"
  export CRAWLER_DIR="\$DATA_DIR/crawler"
  export MITM_DIR="\$DATA_DIR/mitm"
  export OBSERVER_DIR="\$DATA_DIR/observer"
  export QUANTUM_DIR="\$DATA_DIR/quantum"
  export ROOT_SCAN_DIR="\$DATA_DIR/root_scan"
  export FIREBASE_SYNC_DIR="\$DATA_DIR/firebase_sync"
  export FRACTAL_ANTENNA_DIR="\$DATA_DIR/fractal_antenna"
  export VORTICITY_DIR="\$DATA_DIR/vorticity"
  export SYMBOLIC_DIR="\$DATA_DIR/symbolic"
  export GEOMETRIC_DIR="\$DATA_DIR/geometric"
  export PROJECTIVE_DIR="\$DATA_DIR/projective"
  export BIOAETHERIC_DIR="\$DATA_DIR/bioaetheric"
  export LINGOSO_DIR="\$DATA_DIR/lingoso"
  export MARKET_DIR="\$DATA_DIR/market"
  export LAGRANGIAN_DIR="\$DATA_DIR/lagrangian"
  export E8_LATTICE="\$LATTICE_DIR/e8_8d_symbolic.vec"
  export LEECH_LATTICE="\$LATTICE_DIR/leech_24d_symbolic.vec"
  export PRIME_SEQUENCE="\$SYMBOLIC_DIR/prime_sequence.sym"
  export GAUSSIAN_PRIME_SEQUENCE="\$SYMBOLIC_DIR/gaussian_prime.sym"
  export QUANTUM_STATE="\$QUANTUM_DIR/quantum_state.qubit"
  export OBSERVER_INTEGRAL="\$OBSERVER_DIR/observer_integral.proj"
  export ROOT_SIGNATURE_LOG="\$ROOT_SCAN_DIR/signatures.log"
  export CRAWLER_DB="\$CRAWLER_DIR/crawler.db"
  export AUTOPILOT_FILE="\$BASE_DIR/.autopilot_enabled"
  export BRAINWORM_DRIVER_FILE="\$BASE_DIR/.rfk_brainworm/driver.sh"
  export ARC_LENGTH_LOG="\$DATA_DIR/arc_length_coherence.log"
  export NATALIA_FIBRATION_LOG="\$DATA_DIR/natalia_fibration.log"
  export LINGOSO_TRAJECTORY_LOG="\$LINGOSO_DIR/trajectory.log"
  export MARKET_IMBALANCE_LOG="\$MARKET_DIR/imbalance.log"
  export LAGRANGIAN_DENSITY_LOG="\$LAGRANGIAN_DIR/density.log"

  local t_raw
  t_raw=\$(date +%s)
  local t_sym
```

```
  t_sym=\$(python3 -c "import sympy as sp; print(sp.Integer(\$t_raw))" 2>/dev/null
|| echo "\$t_raw")
  export SESSION_ID=\$(python3 -c "
import sympy as sp, hashlib, os
t= sp.Integer(\$t_raw)
mod_t = t % 1000000
try:
    rand_bytes = os.urandom(16)
except:
    rand_bytes = str(mod_t).encode()
session_id = hashlib.sha256(rand_bytes + str(mod_t).encode()).hexdigest()[:32]
print(session_id)
" 2>/dev/null || echo "fallback_session_\$(printf '%06d' \$((t_raw % 1000000)))")
}

# === FUNCTION: prompt_for_credentials ===
prompt_for_credentials(){
  safe_log "Autonomous credential provisioning (no user prompts)"
  mkdir -p "\$BASE_DIR" 2>/dev/null || { safe_log "Failed to create base
directory"; return 1; }
  local env_local_path="\$BASE_DIR/.env.local"
  if [[ ! -f "\$env_local_path" ]]; then
    touch "\$env_local_path"
    chmod 600 "\$env_local_path"
  fi
  if [[ -s "\$env_local_path" ]]; then
    safe_log "Using existing .env.local credentials"
    return 0
  fi
  local auto_login=""
  local auto_password=""
  if command -v termux-dialog &>/dev/null; then
    auto_login=\$(termux-dialog text -t "Login" -i "crawler" 2>/dev/null | jq -r
'.text//empty' || echo "")
    if [[ -n "\$auto_login" ]]; then
      auto_password=\$(termux-dialog text -t "Password" -i "password" 2>/dev/null |
jq -r '.text//empty' || echo "")
    fi
  fi
  if [[ -z "\$auto_login" ]]; then
    safe_log "No credentials detected; operating in local-only autonomous mode"
    return 0
  fi
  printf -v auto_login_escaped '%q' "\$auto_login"
  printf -v auto_password_escaped '%q' "\$auto_password"
  echo "CRAWLER_LOGIN=\$auto_login_escaped" > "\$env_local_path"
  echo "CRAWLER_PASSWORD=\$auto_password_escaped" >> "\$env_local_path"
  chmod 600 "\$env_local_path"
  safe_log "Autonomous credentials provisioned to .env.local"
}
```

```
# === FUNCTION: detect_hardware_capabilities ===
detect_hardware_capabilities(){
  safe_log "Detecting hardware capabilities for adaptive execution"
  HARDWARE_PROFILE["ARCH"]=\$(uname -m 2>/dev/null || echo "unknown")
  HARDWARE_PROFILE["CPU_CORES"]=\$(nproc 2>/dev/null || echo "1")
  HARDWARE_PROFILE["MEMORY_MB"]=\$(python3 -c "
import sympy as sp
try:
    with open('/proc/meminfo', 'r') as f:
        for line in f:
            if line.startswith('MemTotal:'):
                kb = int(line.split()[1])
                mb = kb // 1024
                print(sp.Integer(mb))
                break
except:
    print(sp.Integer(512))
" 2>/dev/null || echo 512)
  HARDWARE_PROFILE["HAS_GPU"]="false"
  if command -v termux-info &>/dev/null; then
    if termux-info 2>/dev/null | grep -qi
"graphics.*adreno\|graphics.*mali\|graphics.*gpu"; then
      HARDWARE_PROFILE["HAS_GPU"]="true"
    fi
  elif [[ -f "/dev/kgsl-3d0" ]] || [[ -d "/sys/class/kgsl" ]] || [[ -d
"/sys/class/drm" ]]; then
    HARDWARE_PROFILE["HAS_GPU"]="true"
  fi
  HARDWARE_PROFILE["HAS_ACCELERATOR"]="false"
  if [[ -d "/dev/dsp" ]] || [[ -c "/dev/ion" ]] || [[ -c "/dev/cdsp" ]]; then
    HARDWARE_PROFILE["HAS_ACCELERATOR"]="true"
  fi
  HARDWARE_PROFILE["HAS_NPU"]="false"
  if [[ -d "/dev/accel" ]] || [[ -c "/dev/npu" ]] || [[ -d "/sys/class/npu" ]] ||
[[ -d "/sys/class/tpu" ]]; then
    HARDWARE_PROFILE["HAS_NPU"]="true"
  fi
  if command -v parallel &>/dev/null; then
    HARDWARE_PROFILE["PARALLEL_CAPABLE"]="true"
  else
    HARDWARE_PROFILE["PARALLEL_CAPABLE"]="false"
    HARDWARE_PROFILE["MISSING_OPTIONAL_COMMANDS"]+="parallel "
  fi
  safe_log "Hardware detection complete: ARCH=\${HARDWARE_PROFILE["ARCH"]},
CORES=\${HARDWARE_PROFILE["CPU_CORES"]}, GPU=\${HARDWARE_PROFILE["HAS_GPU"]},
NPU=\${HARDWARE_PROFILE["HAS_NPU"]}"
}

# === FUNCTION: install_dependencies ===
```

```
install_dependencies(){
  safe_log "Installing Termux-compatible packages without upgrading pip"
  if ! pkg update -y >/dev/null 2>&1; then
    safe_log "Warning: pkg update failed, continuing with installation"
  fi
  local missing_deps=()
  for pkg in "\${TERMUX_PACKAGES_TO_INSTALL[@]}"; do
    if ! pkg list-installed 2>/dev/null | grep -q "^\${pkg}/"; then
      missing_deps+=("\$pkg")
    fi
  done
  if [[ \${#missing_deps[@]} -gt 0 ]]; then
    if pkg install -y "\${missing_deps[@]}" >/dev/null 2>&1; then
      safe_log "Successfully installed packages: \${missing_deps[*]}"
    else
      safe_log "Failed to install one or more packages: \${missing_deps[*]}"
      return 1
    fi
  else
    safe_log "All Termux packages already installed"
  fi
  safe_log "Python dependencies not installed (using pure bash for web crawling)"
}

# === FUNCTION: init_all_directories ===
init_all_directories(){
  safe_log "Initializing full directory structure"
  local dirs=(
    "\$BASE_DIR" "\$DATA_DIR" "\$HOPF_FIBRATION_DIR" "\$LATTICE_DIR" "\$CORE_DIR"
    "\$CRAWLER_DIR" "\$MITM_DIR" "\$MITM_DIR/certs" "\$MITM_DIR/private"
"\$OBSERVER_DIR"
    "\$QUANTUM_DIR" "\$ROOT_SCAN_DIR" "\$FIREBASE_SYNC_DIR"
"\$FIREBASE_SYNC_DIR/pending"
    "\$FIREBASE_SYNC_DIR/processed" "\$FRACTAL_ANTENNA_DIR" "\$VORTICITY_DIR"
    "\$SYMBOLIC_DIR" "\$GEOMETRIC_DIR" "\$PROJECTIVE_DIR" "\$BIOAETHERIC_DIR"
    "\$LINGOSO_DIR" "\$MARKET_DIR" "\$LAGRANGIAN_DIR" "\$BASE_DIR/.rfk_brainworm"
    "\$BASE_DIR/.rfk_brainworm/output" "\$BASE_DIR/debug" "\$BASE_DIR/backups"
"\$BASE_DIR/tests"
  )
  local failed_dirs=()
  for dir in "\${dirs[@]}"; do
    if ! mkdir -p "\$dir" 2>/dev/null; then
      failed_dirs+=("\$dir")
    fi
  done
  if [[ \${#failed_dirs[@]} -gt 0 ]]; then
    safe_log "Failed to create directories: \${failed_dirs[*]}"
    return 1
  else
    safe_log "Directory and file structure initialized successfully"
```

```
  fi
}

# === FUNCTION: validate_python_environment ===
validate_python_environment(){
  safe_log "Validating Python environment for symbolic computation (Exact
Arithmetic Enforced)"
  if ! python3 -c "
import sympy
min_version = '1.6'
try:
    from packaging.version import parse as vparse
    if vparse(sympy.__version__) < vparse(min_version):
        raise Exception(f'sympy version {sympy.__version__} found, but >=
{min_version} required')
except ImportError:
    pass
print('All required Python packages present')
" 2>/dev/null; then
    safe_log "Python environment validation failed: missing or insufficient sympy.
Attempting fallback."
    if ! python3 -c "import sympy" 2>/dev/null; then
      if pip3 install --no-cache-dir --disable-pip-version-check sympy >/dev/null
2>&1; then
        safe_log "sympy installed via pip. Re-validating."
        if python3 -c "
import sympy
min_version = '1.6'
try:
    from packaging.version import parse as vparse
    if vparse(sympy.__version__) < vparse(min_version):
        raise Exception(f'sympy version {sympy.__version__} found, but >=
{min_version} required')
except ImportError:
    pass
" 2>/dev/null; then
          safe_log "Python environment validated after sympy install."
          return 0
        fi
      fi
    fi
    safe_log "Python symbolic computation validation failed. Sympy is mandatory for
TF compliance."
    return 1
  fi
  safe_log "Python environment validated for symbolic computation (sympy >= 1.6)."
  return 0
}

# === FUNCTION: safe_sympy_eval ===
```

```
safe_sympy_eval(){
  local expr="\$1"
  local result
  result=\$(python3 -c "
import sympy as sp
from sympy import S, sqrt, pi, E, I, zeta, isprime, Rational
try:
    expr= sp.sympify('''\$expr''')
    if 'zeta' in '''\$expr''':
        s= sp.sympify('''\$expr'''.split('zeta(')[1].split(')')[0])
        if sp.re(s)!= S(1)/2: s= S(1)/2+ I*sp.im(s)
        result= zeta(s)
    else:
        result= expr
    print(result)
except Exception as e:
    print('0')
" 2>/dev/null)
  echo "\$result"
}

# === FUNCTION: apply_dbz_logic ===
apply_dbz_logic(){
  local psi_re="\$1"
  local option_a="\$2"
  local option_b="\$3"
  TF_CORE["DBZ_CHOICE_HISTORY"]=\$(( \${TF_CORE["DBZ_CHOICE_HISTORY"]} + 1 ))
  if python3 -c "
import sympy as sp
from sympy import S
try:
    psi_re_val= sp.sympify('''\$psi_re''')
    if psi_re_val.is_real:
        result='''\$option_a''' if psi_re_val > S(0) else '''\$option_b'''
    else:
        result= '''\$option_a''' if sp.re(psi_re_val) > S(0) else '''\$option_b'''
    print(result)
except Exception:
    print('''\$option_b''')
" 2>/dev/null; then
    return 0
  else
    echo "\$option_b"
    return 0
  fi
}

# === FUNCTION: adaptive_leech_lattice_packing ===
adaptive_leech_lattice_packing(){
  safe_log "Adaptive Leech lattice construction: Using pre-generated symbolic
```

```
dataset for Termux/ARM64 compatibility"
  local cpu_cores="\${HARDWARE_PROFILE["CPU_CORES"]}"
  local memory_mb="\${HARDWARE_PROFILE["MEMORY_MB"]}"
  local has_gpu="\${HARDWARE_PROFILE["HAS_GPU"]}"
  local has_npu="\${HARDWARE_PROFILE["HAS_NPU"]}"
  safe_log "Hardware context: \$cpu_cores cores, \$memory_mb MB RAM, GPU=\$has_gpu,
NPU=\$has_npu"
  local vector_limit=100
  if [[ \$memory_mb -ge 2048 ]]; then
    vector_limit=500
  elif [[ \$memory_mb -ge 1024 ]]; then
    vector_limit=250
  fi
  pre_generated_leech_dataset "\$vector_limit"
}

# === FUNCTION: pre_generated_leech_dataset ===
pre_generated_leech_dataset() {
  local vector_limit=\${1:-100}
  safe_log "Loading pre-generated, minimal symbolic Leech lattice dataset (limit:
\$vector_limit vectors)"
  mkdir -p "\$LATTICE_DIR" 2>/dev/null || { safe_log "Failed to create lattice
directory"; return 1; }
  if [[ -f "\$LEECH_LATTICE" ]] && [[ -s "\$LEECH_LATTICE" ]] &&
validate_leech_partial; then
    local current_count=\$(wc -l < "\$LEECH_LATTICE" 2>/dev/null || echo "0")
    if [[ \$current_count -ge \$vector_limit ]]; then
      safe_log "Valid pre-generated Leech lattice found at \$LEECH_LATTICE
(\$current_count vectors)"
      return 0
    fi
  fi
  if python3 -c "
import sympy as sp
from sympy import S, Rational
vectors=[]
for i in range(24):
  for sign in [1,-1]:
    v=[S.Zero]*24
    v[i]= sign*S(4)
    vectors.append(v)
golay_vectors=[
  [Rational(-3,2)]+[Rational(1,2)]*23,
  [Rational(1,2), Rational(-3,2)]+[Rational(1,2)]*22,
  [Rational(1,2)]*2+[Rational(-3,2)]+[Rational(1,2)]*21,
  [Rational(1,2)]*3+[Rational(-3,2)]+[Rational(1,2)]*20,
  [Rational(1,2)]*4+[Rational(-3,2)]+[Rational(1,2)]*19,
  [Rational(1,2)]*5+[Rational(-3,2)]+[Rational(1,2)]*18,
  [Rational(1,2)]*6+[Rational(-3,2)]+[Rational(1,2)]*17,
  [Rational(1,2)]*7+[Rational(-3,2)]+[Rational(1,2)]*16,
```

```
  [Rational(1,2)]*8+[Rational(-3,2)]+[Rational(1,2)]*15,
  [Rational(1,2)]*9+[Rational(-3,2)]+[Rational(1,2)]*14,
  [Rational(1,2)]*10+[Rational(-3,2)]+[Rational(1,2)]*13,
  [Rational(1,2)]*11+[Rational(-3,2)]+[Rational(1,2)]*12
]
vectors.extend(golay_vectors)
unique_vectors=[]
seen=set()
for v in vectors:
  v_tuple=tuple(str(coord) for coord in v)
  if v_tuple not in seen:
    seen.add(v_tuple)
    unique_vectors.append(v)
unique_vectors.sort(key=lambda x: tuple(str(coord) for coord in x[:4]))
final_vectors=unique_vectors[:\$vector_limit]
try:
  with open('\$LEECH_LATTICE', 'w') as f:
    for v in final_vectors:
      f.write(' '.join([str(coord) for coord in v])+'\n')
  print(f'Pre-generated Leech lattice dataset created: {len(final_vectors)}
vectors')
except Exception as e:
  print(f'Error writing Leech lattice: {str(e)}')
  exit(1)
" 2>/dev/null; then
    local vector_count=\$(wc -l < "\$LEECH_LATTICE" 2>/dev/null || echo "0")
    safe_log "Pre-generated Leech lattice dataset loaded: \$vector_count vectors"
    return 0
  else
    safe_log "Failed to create pre-generated Leech lattice dataset"
    return 1
  fi
}

# === FUNCTION: full_leech_construction ===
full_leech_construction(){
  safe_log "Full Leech lattice construction is disabled on Termux. Using pre-
generated dataset."
  pre_generated_leech_dataset
}

# === FUNCTION: segmented_leech_construction ===
segmented_leech_construction(){
  safe_log "Segmented Leech lattice construction is disabled on Termux. Using pre-
generated dataset."
  pre_generated_leech_dataset
}

# === FUNCTION: generate_segment_type1 ===
generate_segment_type1(){
```

```
  safe_log "Segment Type 1 generation is deprecated. Using pre-generated dataset."
  return 1
}

# === FUNCTION: generate_segment_type2 ===
generate_segment_type2(){
  safe_log "Segment Type 2 generation is deprecated. Using pre-generated dataset."
  return 1
}

# === FUNCTION: generate_segment_type3 ===
generate_segment_type3(){
  safe_log "Segment Type 3 generation is deprecated. Using pre-generated dataset."
  return 1
}

# === FUNCTION: validate_leech_partial ===
validate_leech_partial(){
  if [[ ! -s "\$LEECH_LATTICE" ]]; then
    safe_log "Leech lattice file missing or empty"
    return 1
  fi
  if python3 -c "
import sympy as sp
from sympy import S
try:
  with open('\$LEECH_LATTICE', 'r') as f:
    lines=f.readlines()
    if len(lines)==0: exit(1)
    valid_count=0
    total_count=0
    for line in lines:
      line=line.strip()
      if not line or line.startswith('#'): continue
      try:
        vec=[sp.sympify(x) for x in line.split()]
        if len(vec)!=24: continue
        norm_sq=sum(coord**2 for coord in vec)
        if norm_sq!=S(4): continue
        all_int=all(coord.is_integer for coord in vec)
        all_half=all((2*coord).is_integer and not coord.is_integer for coord in
vec)
        if not (all_int or all_half): continue
        total=sum(vec)
        if not total.is_integer or (int(total)%2!=0): continue
        valid_count+=1
        total_count+=1
    if total_count>0 and valid_count==total_count: exit(0)
    else: exit(1)
  except Exception: continue
```

```
except Exception: exit(1)
" 2>/dev/null; then
    safe_log "Leech lattice validation passed: 100% norm, coordinate, and parity compliance"
    return 0
  else
    safe_log "Leech lattice validation failed: Not all vectors satisfy Leech conditions"
    return 1
  fi
}

# === FUNCTION: e8_lattice_packing ===
e8_lattice_packing(){
  safe_log "Constructing E8 root lattice via symbolic representation with adaptive resource control"
  mkdir -p "\$LATTICE_DIR" 2>/dev/null || true
  if [[ -f "\$E8_LATTICE" ]] && [[ -s "\$E8_LATTICE" ]]; then
    if validate_e8; then
      safe_log "Valid E8 lattice found at \$E8_LATTICE"
      return 0
    else
      safe_log "Existing E8 lattice invalid, regenerating"
      rm -f "\$E8_LATTICE" 2>/dev/null || true
    fi
  fi
  local cpu_cores="\${HARDWARE_PROFILE["CPU_CORES"]}"
  local memory_mb="\${HARDWARE_PROFILE["MEMORY_MB"]}"
  local timeout_duration=120
  if [[ "\$memory_mb" -ge 2048 ]] && [[ "\$cpu_cores" -ge 4 ]]; then
    timeout_duration=300
  elif [[ "\$memory_mb" -ge 1024 ]] && [[ "\$cpu_cores" -ge 2 ]]; then
    timeout_duration=180
  fi
  safe_log "E8 construction: timeout=\${timeout_duration}s based on hardware profile"
  if timeout "\$timeout_duration" python3 -c "
import sympy as sp
from sympy import S, Rational
inv2=Rational(1,2)
roots=[]
for i in range(8):
  for j in range(i+1, 8):
    for si in [1,-1]:
      for sj in [1,-1]:
        v=[S.Zero]*8
        v[i]=si*S.One
        v[j]=sj*S.One
        roots.append(v)
from itertools import combinations
```

```
for k in range(0, 9, 2):
  for minus_indices in combinations(range(8), k):
    v=[inv2]*8
    for idx in minus_indices: v[idx]=-inv2
    roots.append(v)
unique_roots=[]
seen=set()
for root in roots:
  v_tuple=tuple(str(coord) for coord in root)
  if v_tuple not in seen:
    seen.add(v_tuple)
    unique_roots.append(root)
unique_roots.sort(key=lambda x: tuple(str(coord) for coord in x))
try:
  with open('\$E8_LATTICE', 'w') as f:
    for v in unique_roots:
      f.write(' '.join([str(coord) for coord in v])+'\n')
  print(f'E8 lattice generated: {len(unique_roots)} roots')
except Exception as e:
  print(f'Error writing E8 lattice: {str(e)}')
  exit(1)
" 2>/dev/null; then
    local count=\$(wc -l < "\$E8_LATTICE" 2>/dev/null || echo "0")
    safe_log "E8 lattice successfully constructed with \$count roots"
    return 0
  else
    safe_log "E8 lattice construction failed or timed out"
    return 1
  fi
}

# === FUNCTION: validate_e8 ===
validate_e8(){
  if [[ ! -s "\$E8_LATTICE" ]]; then
    safe_log "E8 lattice file missing or empty"
    return 1
  fi
  if python3 -c "
import sympy as sp
from sympy import S
try:
  with open('\$E8_LATTICE', 'r') as f:
    lines=f.readlines()
    vectors=[]
    for line in lines:
      line=line.strip()
      if not line or line.startswith('#'): continue
      try:
        vec=[sp.sympify(x.strip()) for x in line.split()]
        if len(vec)==8: vectors.append(vec)
```

```
      except Exception: continue
    if len(vectors)<240: exit(1)
    invalid_count=0
    for v in vectors:
      norm_sq=sum(coord**2 for coord in v)
      if norm_sq!=S(2): invalid_count+=1
    if invalid_count==0: exit(0)
    else: exit(1)
except Exception: exit(1)
" 2>/dev/null; then
    safe_log "E8 lattice validation passed: 100% norm compliance"
    return 0
  else
    safe_log "E8 lattice validation failed: Not all vectors have norm squared= 2"
    return 1
  fi
}

# === FUNCTION: generate_prime_sequence ===
generate_prime_sequence(){
  safe_log "Generating symbolic prime sequence via 6m±1 sieve with exact arithmetic
(Arc-Length Coherent)"
  if [[ -f "\$PRIME_SEQUENCE" ]] && [[ -s "\$PRIME_SEQUENCE" ]]; then
    local count=\$(wc -l < "\$PRIME_SEQUENCE" 2>/dev/null || echo "0")
    if [[ "\$count" -ge 1000 ]]; then
      safe_log "Prime sequence already sufficient: \$count primes"
      return 0
    fi
  fi
  mkdir -p "\$SYMBOLIC_DIR" 2>/dev/null || { safe_log "Failed to create symbolic
directory"; return 1; }
  if python3 -c "
import sympy as sp
from sympy import S, Rational, Integer
primes=[]
n=Integer(2)
target_count=1000
progress_checkpoints={100, 250, 500, 750}
while len(primes)<target_count:
  if sp.isprime(n):
    primes.append(Integer(n))
    if len(primes) in progress_checkpoints:
      print(f'Generated {len(primes)} primes...')
  n+=Integer(1)
  if n>100000: break
try:
  with open('\$PRIME_SEQUENCE', 'w') as f:
    for p in primes: f.write(str(p)+'\n')
  print(f'Generated {len(primes)} symbolic primes')
except Exception as e:
```

```
  print(f'Error writing prime sequence: {str(e)}')
  exit(1)
" 2>/dev/null; then
    local generated_count=\$(wc -l < "\$PRIME_SEQUENCE" 2>/dev/null || echo "0")
    safe_log "Generated \$generated_count symbolic primes (Exact Integer Type)"
    return 0
  else
    safe_log "Failed to generate symbolic prime sequence"
    return 1
  fi
}

# === FUNCTION: generate_gaussian_primes ===
generate_gaussian_primes(){
  safe_log "Generating Gaussian primes via symbolic norm classification
(algorithmic, exact)"
  if [[ -f "\$GAUSSIAN_PRIME_SEQUENCE" ]] && [[ -s "\$GAUSSIAN_PRIME_SEQUENCE" ]];
then
    local count=\$(wc -l < "\$GAUSSIAN_PRIME_SEQUENCE" 2>/dev/null || echo "0")
    if [[ "\$count" -ge 500 ]]; then
      safe_log "Gaussian prime sequence already sufficient: \$count primes"
      return 0
    fi
  fi
  mkdir -p "\$SYMBOLIC_DIR" 2>/dev/null || { safe_log "Failed to create symbolic
directory"; return 1; }
  if python3 -c "
import sympy as sp
from sympy import S, I, Integer
gaussian_primes=[]
limit=30
for a in range(-limit, limit+1):
  for b in range(-limit, limit+1):
    if a==0 and b==0: continue
    norm_sq=a*a+b*b
    if a==0:
      if b!=0 and sp.isprime(abs(b)) and (abs(b)%4==3):
        gaussian_primes.append((Integer(a), Integer(b)))
    elif b==0:
      if a!=0 and sp.isprime(abs(a)) and (abs(a)%4==3):
        gaussian_primes.append((Integer(a), Integer(b)))
    else:
      if sp.isprime(norm_sq):
        gaussian_primes.append((Integer(a), Integer(b)))
seen=set()
unique_primes=[]
for gp in gaussian_primes:
  if gp not in seen:
    seen.add(gp)
    unique_primes.append(gp)
```

```
unique_primes.sort(key=lambda x: (x[0]**2+x[1]**2, x[0], x[1]))
final_primes=unique_primes[:500]
try:
  with open('\$GAUSSIAN_PRIME_SEQUENCE', 'w') as f:
    for a, b in final_primes: f.write(f'{a} {b}\n')
  print(f'Generated {len(final_primes)} symbolic Gaussian primes')
except Exception as e:
  print(f'Error writing Gaussian primes: {str(e)}')
  exit(1)
" 2>/dev/null; then
    local generated_count=\$(wc -l < "\$GAUSSIAN_PRIME_SEQUENCE" 2>/dev/null ||
echo "0")
    safe_log "Generated \$generated_count symbolic Gaussian primes (Exact Integer
Type)"
    return 0
  else
    safe_log "Failed to generate Gaussian primes"
    return 1
  fi
}

# === FUNCTION: dbz_resample_zeta_s ===
dbz_resample_zeta_s() {
  local s_raw="\$1"
  python3 -c "
import sympy as sp
from sympy import S, I
s=sp.sympify('''\$s_raw''')
if sp.re(s)!=S(1)/2: s=S(1)/2+I*sp.im(s)
print(s)
"
}

# === FUNCTION: generate_quantum_state ===
generate_quantum_state(){
  safe_log "Generating symbolically exact quantum state via Riemann zeta critical
line enforcement"
  mkdir -p "\$QUANTUM_DIR" 2>/dev/null || { safe_log "Failed to create quantum
directory"; return 1; }
  local t_raw=\$(date +%s)
  local t_sym=\$(python3 -c "import sympy as sp; print(sp.Integer(\$t_raw))"
2>/dev/null || echo "\$t_raw")
  local t_mod=\$(python3 -c "import sympy as sp; t= sp.Integer(\$t_raw);
print(int(t%1000))" 2>/dev/null || echo "0")
  local s_dbz=\$(dbz_resample_zeta_s "S(1)/2+I*\$t_mod")
  if python3 -c "
import sympy as sp
from sympy import S, I, pi, sqrt, exp, zeta, symbols, Rational
t= sp.Integer(\$t_raw)
s= sp.sympify('''\$s_dbz''')
```

```
try:
  zeta_s=zeta(s)
except Exception as e:
  s=S(1)/2+I*sp.im(s)
  try: zeta_s=zeta(s)
  except Exception as e2: zeta_s=sp.Function('zeta')(s)
modulation=S(1)
try:
  with open('\$LEECH_LATTICE', 'r') as f:
    lines=f.readlines()
    if lines:
      first_line=lines[0].strip()
      if first_line:
        vec=[sp.sympify(x) for x in first_line.split()]
        if len(vec)==24:
          norm_sq=sum(coord**2 for coord in vec)
          if norm_sq==S(4): modulation=norm_sq/S(4)
        else:
          total_norm=sum(sp.sqrt(sum(coord**2 for coord in v)) for v in
[[sp.sympify(x) for x in line.split()] for line in lines if line.strip()])
          if total_norm!=S.Zero:
            probabilities=[sp.sqrt(sum(coord**2 for coord in v))/total_norm for v
in [[sp.sympify(x) for x in line.split()] for line in lines if line.strip()]]
            entropy=-sum(p*sp.log(p) for p in probabilities if p!=S.Zero)
            modulation=entropy/S(10)
except Exception as e: pass
try:
  modulus=sp.Abs(zeta_s)
  psi=(zeta_s/(S(1)+modulus))*modulation
except Exception as e: psi=(zeta_s/(S(1)+sqrt(S(2))))*modulation
psi_re=sp.re(psi)
psi_im=sp.im(psi)
try:
  with open('\$QUANTUM_STATE', 'w') as f:
    f.write('{\"real\":\"'+str(psi_re)+'\",\"imag\":\"'+str(psi_im)+'\"}\n')
  print('Quantum state generated symbolically')
except Exception as e:
  print(f'Error writing quantum state: {str(e)}')
  exit(1)
" 2>/dev/null; then
    safe_log "Quantum state generated: symbolic Ψ(s)= ζ(s)/(1+|ζ(s)|)*modulation on
Re(s)=1/2"
    return 0
  else
    safe_log "Failed to generate symbolic quantum state"
    return 1
  fi
}

# === FUNCTION: generate_observer_integral ===
```

```
generate_observer_integral(){
  safe_log "Generating observer integral Φ=Q(s)=(s, ζ(s), ζ(s+1), ζ(s+2)) in exact symbolic form"
  mkdir -p "\$OBSERVER_DIR" 2>/dev/null || { safe_log "Failed to create observer directory"; return 1; }
  local t_raw=\$(date +%s)
  local t_sym=\$(python3 -c "import sympy as sp; print(sp.Integer(\$t_raw))" 2>/dev/null || echo "\$t_raw")
  local t_mod=\$(python3 -c "import sympy as sp; t= sp.Integer(\$t_raw); print(int(t%1000))" 2>/dev/null || echo "0")
  local s_base=\$(dbz_resample_zeta_s "S(1)/2+I*\$t_mod")
  if python3 -c "
import sympy as sp
from sympy import S, I, zeta, sqrt, pi, Rational
s= sp.sympify('''\$s_base''')
components=[]
for shift in [0, 1, 2]:
  s_shifted=s+shift
  if sp.re(s_shifted)!=S(1)/2: s_shifted=S(1)/2+I*sp.im(s_shifted)
  try: zeta_val=zeta(s_shifted)
  except Exception as e: zeta_val=sp.Function('zeta')(s_shifted)
  components.append(zeta_val)
components.insert(0, s)
Phi_real=sum(sp.re(c) for c in components)
Phi_imag=sum(sp.im(c) for c in components)
Phi_real=Phi_real*Rational(1,10)
Phi_imag=Phi_imag*Rational(1,10)
try:
  with open('\$FRACTAL_ANTENNA_DIR/antenna_state.sym', 'r') as f:
    antenna_state=f.read().strip()
    if antenna_state:
      antenna_val=sp.sympify(antenna_state)
      Phi_real=Phi_real*antenna_val
      Phi_imag=Phi_imag*antenna_val
except Exception as e: pass
try:
  with open('\$OBSERVER_INTEGRAL', 'w') as f:
    f.write('{\"real\":\"'+str(Phi_real)+'\",\"imag\":\"'+str(Phi_imag)+'\"}\n')
  print('Observer integral generated symbolically')
except Exception as e:
  print(f'Error writing observer integral: {str(e)}')
  exit(1)
" 2>/dev/null; then
    safe_log "Observer integral generated: Φ= Σ Re/Im of(s, ζ(s), ζ(s+1), ζ(s+2)) modulated by fractal antenna"
    return 0
  else
    safe_log "Failed to generate symbolic observer integral"
    return 1
  fi
```

```
}

# === FUNCTION: measure_consciousness ===
measure_consciousness(){
  safe_log "Measuring consciousness via symbolic observer operator O[Ψ]= ∫ Ψ† · Φ · Ψ d⁴q with vorticity feedback"
  local prime_count=\$(wc -l < "\$PRIME_SEQUENCE" 2>/dev/null || echo "0")
  local p_max=\$(tail -n 1 "\$PRIME_SEQUENCE" 2>/dev/null || echo "2")
  local valid_pairs=\$(wc -l < "\$CORE_DIR/prime_lattice_map.sym" 2>/dev/null ||
echo "0")
  local total_primes=\$(python3 -c "print(max(\$prime_count, 1))" 2>/dev/null ||
echo "1")
  local t_raw=\$(date +%s)
  local t_sym=\$(python3 -c "import sympy as sp; print(sp.Integer(\$t_raw))"
2>/dev/null || echo "\$t_raw")

  mkdir -p "\$BASE_DIR" 2>/dev/null || { safe_log "Failed to create base
directory"; return 1; }

  local dbz_history="\${TF_CORE["DBZ_CHOICE_HISTORY"]:-0}"
  if python3 -c "
import sympy as sp
from sympy import S, pi, log, sqrt, exp, li, Abs, symbols, Rational
x_sym= symbols('x')
C= S(1)
alignment= sp.Rational(\$valid_pairs, max(\$total_primes, 1))
pi_x= sp.Integer(\$prime_count)
Li_x= li(x_sym)
try:
  Delta_x= Abs(pi_x- Li_x.subs(x_sym, sp.Integer(\$p_max)))
except Exception as e: Delta_x= Abs(pi_x- sp.log(sp.Integer(\$p_max)))
try:
  sqrt_x= sqrt(sp.Integer(\$t_raw))
  log_x= log(sp.Integer(\$t_raw)+ 1)
  denom= C* sqrt_x* log_x
  if denom!= 0: scaled_Delta= Delta_x/ denom; riemann_factor= exp(-scaled_Delta)
  else: riemann_factor= S(0)
except Exception as e: riemann_factor= S(0)
try:
  phi_data= open('\$OBSERVER_INTEGRAL', 'r').read().strip()
  import json
  phi_json= json.loads(phi_data)
  phi_real= sp.sympify(phi_json['real'])
  phi_imag= sp.sympify(phi_json['imag'])
  Phi= phi_real+ sp.I* phi_imag
  aetheric_stability= Abs(Phi)
except Exception as e: aetheric_stability= S(1)
vorticity= S(1)
try:
  current_phi_real= phi_real
```

```
  current_phi_imag= phi_imag
  prev_phi_file= '\$VORTICITY_DIR/prev_phi.sym'
  if sp.simplify(current_phi_real)!= S(0) or sp.simplify(current_phi_imag)!= S(0):
    try:
      with open(prev_phi_file, 'r') as f:
        prev_data= f.read().strip().split()
        if len(prev_data)== 2:
          prev_phi_real= sp.sympify(prev_data[0])
          prev_phi_imag= sp.sympify(prev_data[1])
          delta_phi_real= current_phi_real- prev_phi_real
          delta_phi_imag= current_phi_imag- prev_phi_imag
          vorticity= sp.sqrt(delta_phi_real**2+ delta_phi_imag**2)
    except Exception as e: vorticity= S(1)
    with open(prev_phi_file, 'w') as f:
      f.write(f'{current_phi_real} {current_phi_imag}\n')
except Exception as e: vorticity= S(1)
dbz_history= int('\$dbz_history')
dbz_influence= S(dbz_history)/ S(100)
I= alignment* riemann_factor* aetheric_stability* vorticity* (S(1)+ dbz_influence)
try:
  psi_data= open('\$QUANTUM_STATE', 'r').read().strip()
  psi_json= json.loads(psi_data)
  psi_real= sp.sympify(psi_json['real'])
  psi_imag= sp.sympify(psi_json['imag'])
  psi= psi_real+ sp.I* psi_imag
  psi_dag= psi_real- sp.I* psi_imag
  integrand= psi_dag* Phi* psi
  observer_operator= integrand
  with open('\$OBSERVER_DIR/observer_operator.sym', 'w') as f:
    f.write(str(observer_operator)+ '\n')
except Exception as e: observer_operator= S(1)
I_final= I* observer_operator
try:
  with open('\$BASE_DIR/consciousness_metric.txt', 'w') as f:
    f.write(str(I_final)+ '\n')
  print(f'Consciousness metric:{I_final}')
except Exception as e:
  print(f'Error writing consciousness metric:{str(e)}')
  exit(1)
" 2>/dev/null; then
    safe_log "Consciousness metric computed symbolically with vorticity and
observer operator"
    return 0
  else
    safe_log "Consciousness metric computation failed"
    return 1
  fi
}

# === FUNCTION: project_prime_to_lattice ===
```

```
project_prime_to_lattice(){
  safe_log "Projecting symbolic prime onto Leech lattice using zeta-driven
minimization with Arc-Length Coherence"
  local p_n=\$(tail -n 1 "\$PRIME_SEQUENCE" 2>/dev/null || echo "2")
  if [[ -z "\$p_n" ]] || [[ "\$p_n"=="2" && \$(wc -l < "\$PRIME_SEQUENCE"
2>/dev/null || echo "0") -le 1 ]]; then
    safe_log "No valid prime to project"
    return 0
  fi
  if ! symbolic_geometry_binding; then
    safe_log "Geometry binding failed, cannot project prime"
    return 1
  fi
  local v_k_str=\$(cat "\$CORE_DIR/projected_vector.vec" 2>/dev/null || echo "")
  local v_k_hash=\$(cat "\$CORE_DIR/projected_vector.hash" 2>/dev/null || echo "")
  if [[ -n "\$v_k_str" ]] && [[ -n "\$v_k_hash" ]]; then
    echo "\$v_k_str" > "\$CORE_DIR/prime_lattice_map.sym"
    echo "PRIME=\$p_n VECTOR_HASH=\$v_k_hash TIMESTAMP=\$(date +%s)
ARC_LENGTH_COHERENCE=verified" >> "\$DNA_LOG"
    safe_log "Prime \$p_n projected to Leech vector \${v_k_hash:0:16}...(Arc-Length
Validated)"
  else
    safe_log "Projection failed: no valid vector"
    return 1
  fi
}

# === FUNCTION: calculate_lattice_entropy ===
calculate_lattice_entropy(){
  safe_log "Calculating lattice entropy via exact norm distribution in Leech
lattice (Symbolic Exactness Enforced)"
  if [[ ! -s "\$LEECH_LATTICE" ]]; then
    safe_log "Leech lattice file missing or empty"
    return 1
  fi
  if python3 -c "
import sympy as sp
from sympy import S, sqrt, log, Rational
try:
  with open('\$LEECH_LATTICE', 'r') as f:
    lines= f.readlines()
  vectors=[]
  for line in lines:
    line= line.strip()
    if not line or line.startswith('#'): continue
    try:
      vec=[sp.sympify(x) for x in line.split()]
      if len(vec)== 24: vectors.append(vec)
    except Exception: pass
  if not vectors: raise ValueError('Empty lattice')
```

```
  norms=[sp.sqrt(sum(coord**2 for coord in v)) for v in vectors]
  total_norm= sum(norms)
  if total_norm== S.Zero: entropy= S.Zero
  else:
    probabilities=[n/ total_norm for n in norms]
    entropy= -sum(p* sp.log(p) for p in probabilities if p!= S.Zero)
  with open('\$LATTICE_DIR/entropy.log', 'w') as f:
    f.write(str(entropy)+ '\n')
  print(f'Lattice entropy computed:{entropy}')
except Exception as e:
  with open('\$LATTICE_DIR/entropy.log', 'w') as f: f.write('0\n')
  print(f'Error computing entropy:{str(e)}')
  exit(1)
" 2>/dev/null; then
    safe_log "Lattice entropy computed symbolically"
    return 0
  else
    safe_log "Lattice entropy computation failed"
    return 1
  fi
}

# === FUNCTION: get_kissing_number ===
get_kissing_number(){
  if [[ ! -f "\$LEECH_LATTICE" ]]; then echo "196560"; return; fi
  local count=0
  while IFS= read -r line || [[ -n "\$line" ]]; do
    line=\$(echo "\$line" | tr -d '\r\n')
    [[ -z "\$line" || "\$line" =~ ^# ]] && continue
    ((count++))
  done < "\$LEECH_LATTICE"
  echo "\$count"
}

# === FUNCTION: optimize_kissing_number ===
optimize_kissing_number(){
  safe_log "Optimizing kissing number via symbolic Delaunay triangulation (Arc-
Length Coherent)"
  local current_kissing=\$(get_kissing_number)
  if [[ \$current_kissing -ge 196560 ]]; then
    safe_log "Kissing number already sufficient: \$current_kissing"
    return 0
  fi
  if python3 -c "
import sympy as sp
from sympy import S, sqrt, pi, Rational
try:
  with open('\$LEECH_LATTICE', 'r') as f:
    lines= f.readlines()
  vectors=[]
```

```
  for line in lines:
    line= line.strip()
    if not line or line.startswith('#'): continue
    try:
      vec=[sp.sympify(x) for x in line.split()]
      if len(vec)== 24: vectors.append(vec)
    except Exception as e: pass
  if len(vectors)>= 196560: exit(0)
  new_vectors=[]
  phi=(S(1)+ sqrt(5))/ S(2)
  for v in vectors[:100]:
    for scale_factor in [Rational(1,2), Rational(2,3), phi/S(3)]:
      new_v=[scale_factor* coord for coord in v]
      new_vectors.append(new_v)
  unique_new=[]
  seen=set()
  for v in new_vectors:
    v_tuple= tuple(str(coord) for coord in v)
    if v_tuple not in seen: seen.add(v_tuple); unique_new.append(v)
  final_new=[]
  for v in unique_new:
    norm_sq= sum(coord**2 for coord in v)
    if norm_sq== S(4): final_new.append(v)
    else:
      if norm_sq!= S.Zero:
        target_norm= S(2)
        current_norm= sp.sqrt(norm_sq)
        scaling_factor= target_norm/ current_norm
        scaled_v=[coord* scaling_factor for coord in v]
        final_new.append(scaled_v)
  with open('\$LEECH_LATTICE', 'a') as f:
    for v in final_new: f.write(','.join([str(coord) for coord in v])+ '\n')
  print(f'Added{len(final_new)} norm-compliant symbolic vectors to optimize kissing
number')
except Exception as e:
  print(f'Kissing optimization failed:{str(e)}')
  exit(1)
" 2>/dev/null; then
    safe_log "Kissing number optimization complete"
    return 0
  else
    safe_log "Kissing optimization failed"
    return 1
  fi
}

# === FUNCTION: resample_zeta_zeros ===
resample_zeta_zeros(){
  safe_log "Applying DbZ resampling: enforcing Re(s)= 1/2 for all zeta zeros
symbolically (Arc-Length Axiom)"
```

```
  mkdir -p "\$SYMBOLIC_DIR" 2>/dev/null || { safe_log "Failed to create symbolic
directory"; return 1; }
  local zero_file="\$SYMBOLIC_DIR/zeta_zeros.sym"
  if [[ -f "\$zero_file" ]] && [[ -s "\$zero_file" ]]; then
    local count=\$(wc -l < "\$zero_file" 2>/dev/null || echo "0")
    if [[ "\$count" -ge 10 ]]; then
      safe_log "Zeta zeros already resampled: \$count zeros"
      return 0
    fi
  fi
  if python3 -c "
import sympy as sp
from sympy import S, I, Symbol
zeros=[]
for k in range(1, 11):
  im_part= Symbol(f'gamma_{k}')
  s= S(1)/2+ I* im_part
  zeros.append(s)
try:
  with open('\$zero_file', 'w') as f:
    for s in zeros: f.write(str(s)+ '\n')
  print('DbZ resampling complete: 10 symbolic zeros with Re(s)=1/2 (exact
placeholders)')
except Exception as e:
  print(f'Error writing zeta zeros:{str(e)}')
  exit(1)
" 2>/dev/null; then
    safe_log "DbZ resampling complete: 10 zeta zeros with Re(s)=1/2 enforced
(symbolic placeholders)"
    return 0
  else
    safe_log "DbZ resampling failed"
    return 1
  fi
}

# === FUNCTION: validate_hopf_continuity ===
validate_hopf_continuity(){
  local quat_file="\${1:-\$HOPF_FIBRATION_DIR/latest.quat}"
  if [[ ! -f "\$quat_file" ]]; then
    safe_log "Hopf fibration file missing: \$quat_file"
    return 1
  fi
  if python3 -c "
import sympy as sp
from sympy import S, sqrt
try:
  with open('\$quat_file', 'r') as f:
    line= f.readline().strip()
    if not line or line.startswith('#'): exit(1)
```

```
    parts= line.split()
    if len(parts)!= 4: exit(1)
    q0= sp.sympify(parts[0])
    q1= sp.sympify(parts[1])
    q2= sp.sympify(parts[2])
    q3= sp.sympify(parts[3])
    norm_sq= q0**2+ q1**2+ q2**2+ q3**2
    if norm_sq== S(1): exit(0)
    else: exit(1)
except Exception as e: exit(1)
" 2>/dev/null; then
    safe_log "Hopf fibration continuity validated: ||q||²= 1 exactly (Arc-Length
Coherent)"
    return 0
  else
    safe_log "Hopf fibration validation failed: ||q||²!= 1"
    return 1
  fi
}

# === FUNCTION: generate_hopf_fibration ===
generate_hopf_fibration(){
  safe_log "Generating symbolic Hopf fibration state via Natalia's Fibrations
(BFAC-to-Z-pinch transformation)"
  mkdir -p "\$HOPF_FIBRATION_DIR" 2>/dev/null || { safe_log "Failed to create Hopf
fibration directory"; return 1; }
  local t_raw=\$(date +%s)
  local t_sym=\$(python3 -c "import sympy as sp; print(sp.Integer(\$t_raw))"
2>/dev/null || echo "\$t_raw")
  local t_mod=\$(python3 -c "import sympy as sp; t= sp.Integer(\$t_raw);
print(int(t%1000))" 2>/dev/null || echo "0")
  local quat_file="\$HOPF_FIBRATION_DIR/hopf_\${t_mod}.quat"
  if python3 -c "
import sympy as sp
from sympy import S, sqrt, Rational
a, b, c, d= sp.symbols('a b c d', real=True)
t_val= sp.Integer(\$t_raw)
a_val= Rational(t_val% 1000, 1000)
b_val= Rational((t_val* 3)% 1000, 1000)
c_val= Rational((t_val* 7)% 1000, 1000)
d_val= Rational((t_val* 11)% 1000, 1000)
q0, q1, q2, q3= a_val, b_val, c_val, d_val
norm_sq= q0**2+ q1**2+ q2**2+ q3**2
if norm_sq!= S(1):
  norm= sp.sqrt(norm_sq)
  q0, q1, q2, q3= q0/norm, q1/norm, q2/norm, q3/norm
epsilon= Rational(1, 100)
perturbation_sum= S.Zero
current_epsilon_power= epsilon
for k in range(1, 11):
```

```
  natalia_k=(t_val% 1000)**k
  term= current_epsilon_power* natalia_k
  perturbation_sum+= term
  if abs(term)< Rational(1, 10**50): break
  current_epsilon_power*= epsilon
q0= q0+ perturbation_sum
try:
  with open('\$quat_file', 'w') as f: f.write(f'{q0} {q1} {q2} {q3}\n')
  with open('\$HOPF_FIBRATION_DIR/latest.quat', 'w') as f: f.write(f'{q0} {q1} {q2}
{q3}\n')
  with open('\$NATALIA_FIBRATION_LOG', 'a') as f: f.write(f'Timestamp:{t_raw},
Perturbation:{perturbation_sum}\n')
  print('Hopf fibration generated symbolically with Natalia perturbation')
except Exception as e:
  print(f'Error writing Hopf fibration:{str(e)}')
  exit(1)
" 2>/dev/null; then
    safe_log "Hopf fibration state generated: \$quat_file (Natalia's Fibrations
Applied)"
    return 0
  else
    safe_log "Failed to generate symbolic Hopf fibration"
    return 1
  fi
}

# === FUNCTION: encode_lingoso_syllable ===
encode_lingoso_syllable(){
  safe_log "Encoding Lingoso phonosyllabic geometry (TF Appendix G)"
  mkdir -p "\$LINGOSO_DIR" 2>/dev/null || { safe_log "Failed to create Lingoso
directory"; return 1; }
  local syllable="\${1:-A}"
  local trajectory_file="\$LINGOSO_DIR/trajectory_\${syllable}.sym"
  if python3 -c "
import sympy as sp
from sympy import S, sqrt, pi, Rational, I
syllable= '\$syllable'
t= sp.Symbol('t')
phi=(S(1)+ sqrt(5))/ S(2)
if syllable== 'A': r= sp.exp(t/ phi); theta= t
elif syllable== 'U': r= S(1); theta= pi* t
elif syllable== 'M': r= S(1); theta= 2* pi* t
else: r= S(1); theta= t
q0= sp.cos(theta/ 2)
q1= sp.sin(theta/ 2)* sp.cos(r)
q2= sp.sin(theta/ 2)* sp.sin(r)
q3= S(0)
try:
  with open('\$trajectory_file', 'w') as f:
    f.write(f'SYLLABLE={syllable}\n')
```

```
    f.write(f'Q0={q0}\n')
    f.write(f'Q1={q1}\n')
    f.write(f'Q2={q2}\n')
    f.write(f'Q3={q3}\n')
  print(f'Lingoso syllable{syllable} encoded symbolically')
except Exception as e:
  print(f'Error encoding Lingoso:{str(e)}')
  exit(1)
" 2>/dev/null; then
    safe_log "Lingoso syllable '\$syllable' encoded to \$trajectory_file"
    return 0
  else
    safe_log "Failed to encode Lingoso syllable"
    return 1
  fi
}

# === FUNCTION: compute_market_imbalance ===
compute_market_imbalance(){
  safe_log "Computing Market Topology imbalance (TF Section 8.4)"
  mkdir -p "\$MARKET_DIR" 2>/dev/null || { safe_log "Failed to create Market
directory"; return 1; }
  local imbalance_file="\$MARKET_DIR/imbalance.log"
  if python3 -c "
import sympy as sp
from sympy import S, Rational
threshold_over= Rational(666, 10)
threshold_under= Rational(333, 10)
state_vector=[Rational(50)]* 5
m= 0
n= 0
for val in state_vector:
  if val> threshold_over: m+= 1
  elif val< threshold_under: n+= 1
imbalance= S(0)
if (m- 1)> (n+ 1): imbalance= S(1)
try:
  with open('\$imbalance_file', 'w') as f:
    f.write(f'IMBALANCE={imbalance}\n')
    f.write(f'BULLISH={m}\n')
    f.write(f'BEARISH={n}\n')
  print(f'Market imbalance computed:{imbalance}')
except Exception as e:
  print(f'Error computing market imbalance:{str(e)}')
  exit(1)
" 2>/dev/null; then
    safe_log "Market topology imbalance computed symbolically"
    return 0
  else
    safe_log "Failed to compute market imbalance"
```

```
    return 1
  fi
}

# === FUNCTION: compute_lagrangian_density ===
compute_lagrangian_density(){
  safe_log "Computing Unified Lagrangian Density (TF Appendix H)"
  mkdir -p "\$LAGRANGIAN_DIR" 2>/dev/null || { safe_log "Failed to create
Lagrangian directory"; return 1; }
  local density_file="\$LAGRANGIAN_DIR/density.log"
  if python3 -c "
import sympy as sp
from sympy import S, Rational, sqrt, pi, symbols, diff
t= symbols('t')
Phi= sp.exp(-t**2)
dPhi= diff(Phi, t)
kinetic= S(1)/2* dPhi**2
lambda_const= Rational(1, 4)
potential= lambda_const/ S(24)* (Phi* Phi)**2
L= kinetic+ potential
try:
  with open('\$density_file', 'w') as f:
    f.write(f'LAGRANGIAN={L}\n')
    f.write(f'KINETIC={kinetic}\n')
    f.write(f'POTENTIAL={potential}\n')
  print(f'Lagrangian density computed:{L}')
except Exception as e:
  print(f'Error computing Lagrangian density:{str(e)}')
  exit(1)
" 2>/dev/null; then
    safe_log "Unified Lagrangian density computed symbolically"
    return 0
  else
    safe_log "Failed to compute Lagrangian density"
    return 1
  fi
}

# === FUNCTION: generate_hw_signature ===
generate_hw_signature(){
  safe_log "Generating symbolic hardware DNA signature with Hopf fibration binding
(Arc-Length Validated)"
  local hw_info=""
  if command -v getprop &>/dev/null; then
    hw_info+=\$(getprop ro.product.manufacturer 2>/dev/null || echo "unknown")
    hw_info+=\$(getprop ro.product.model 2>/dev/null || echo "unknown")
    hw_info+=\$(getprop ro.build.version.release 2>/dev/null || echo "unknown")
    hw_info+=\$(settings get secure android_id 2>/dev/null || openssl rand -hex 16)
  else
    hw_info+=\$(uname -o 2>/dev/null || echo "unknown")
```

```
    hw_info+=\$(uname -m 2>/dev/null || echo "unknown")
    hw_info+=\$(uname -r 2>/dev/null || echo "unknown")
    hw_info+=\$(cat /etc/machine-id 2>/dev/null || openssl rand -hex 16)
  fi
  hw_info+=\$(cat /proc/cpuinfo 2>/dev/null | grep 'Serial' | cut -d ':' -f2 | tr -
d ' ' || echo "no_serial")
  local raw_hash=\$(echo -n "\$hw_info" | sha256sum | cut -d ' ' -f1)
  local latest_hopf=\$(ls -t "\$HOPF_FIBRATION_DIR"/hopf_*.quat 2>/dev/null | head
-n1)
  local hopf_state="1/2 0 0 sqrt(3)/2"
  if [[ -f "\$latest_hopf" ]]; then
    read -r hopf_state < "\$latest_hopf"
  else
    if ! generate_hopf_fibration; then
      safe_log "Failed to generate Hopf fibration for HW signature"
      return 1
    fi
    latest_hopf=\$(ls -t "\$HOPF_FIBRATION_DIR"/hopf_*.quat 2>/dev/null | head -n1)
    [[ -f "\$latest_hopf" ]] && read -r hopf_state < "\$latest_hopf"
  fi
  if python3 -c "
import sympy as sp
from sympy import S, sqrt, pi
hopf_str= '\$hopf_state'
parts= hopf_str.split()
if len(parts)== 4:
  q0= sp.sympify(parts[0])
  q1= sp.sympify(parts[1])
  q2= sp.sympify(parts[2])
  q3= sp.sympify(parts[3])
else:
  q0, q1, q2, q3= S(1)/2, S(0), S(0), sqrt(3)/2
norm_sq= q0**2+ q1**2+ q2**2+ q3**2
if norm_sq!= S(1):
  norm= sp.sqrt(norm_sq)
  q0, q1, q2, q3= q0/norm, q1/norm, q2/norm, q3/norm
weight=(q0+ q1+ q2+ q3)/ S(4)
phi_expr= sp.sympify('\$PHI_SYMBOLIC')
influence= sp.Mod(weight* phi_expr, S(1))
influence_str= str(influence)
import hashlib
h= hashlib.sha512()
h.update('\$raw_hash'.encode('utf-8'))
h.update(influence_str.encode('utf-8'))
signature= h.hexdigest()
try:
  with open('\$BASE_DIR/.hw_dna', 'w') as f:
    f.write(signature+ '\n')
  print(f'Hardware DNA:{signature[:16]}...')
except Exception as e:
```

```
  print(f'Error writing hardware DNA:{str(e)}')
  exit(1)
" 2>/dev/null; then
    safe_log "Hardware DNA (Hopf-Validated): \$(head -c 16
"\$BASE_DIR/.hw_dna")..."
    return 0
  else
    safe_log "Failed to generate symbolic hardware signature"
    return 1
  fi
}

# === FUNCTION: root_scan_init ===
root_scan_init(){
  safe_log "Initializing symbolic root scan subsystem with prime-lattice alignment"
  mkdir -p "\$ROOT_SCAN_DIR" 2>/dev/null || { safe_log "Failed to create root scan
directory"; return 1; }
  if [[ ! -f "\$ROOT_SIGNATURE_LOG" ]]; then
    touch "\$ROOT_SIGNATURE_LOG" || safe_log "Warning: Could not create signature
log"
  fi
  if [[ -f "\$CORE_DIR/prime_lattice_map.sym" ]] && [[ -f "\$PRIME_SEQUENCE" ]];
then
    local valid_pairs=\$(wc -l < "\$CORE_DIR/prime_lattice_map.sym" 2>/dev/null ||
echo "0")
    local total_primes=\$(wc -l < "\$PRIME_SEQUENCE" 2>/dev/null || echo "1")
    if python3 -c "
import sympy as sp
from sympy import S, sqrt, pi
alignment= sp.Rational(\$valid_pairs, \$total_primes)
phi= sp.sympify('\$PHI_SYMBOLIC')
modulated= sp.Mod(alignment* phi, S(1))
mod_str= str(modulated)
import hashlib
h= hashlib.sha256()
h.update(mod_str.encode('utf-8'))
signature= h.hexdigest()
while len(signature)< 32: signature= '0'+ signature
with open('\$ROOT_SIGNATURE_LOG', 'w') as f: f.write(signature+ '\n')
print(f'Root signature generated:{signature[:24]}...')
" 2>/dev/null; then
      safe_log "Root signature generated from symbolic alignment"
    else
      safe_log "Failed to generate symbolic root signature"
      return 1
    fi
  else
    safe_log "Insufficient symbolic data for root signature"
  fi
  safe_log "Root scan subsystem initialized"
```

```
}

# === FUNCTION: validate_fractal_antenna ===
validate_fractal_antenna(){
  local antenna_file="\${1:-\$FRACTAL_ANTENNA_DIR/antenna_state.sym}"
  if [[ ! -f "\$antenna_file" ]]; then
    safe_log "Fractal antenna state file missing: \$antenna_file"
    return 1
  fi
  local state=\$(cat "\$antenna_file" 2>/dev/null | tr -d '[:space:]')
  if [[ -z "\$state" ]] || [[ "\$state"=="0" ]] || [[ "\$state"=="S(0)" ]]; then
    safe_log "Fractal antenna state invalid: \$state"
    return 1
  fi
  safe_log "Fractal antenna state validated: \$antenna_file"
  return 0
}

# === FUNCTION: validate_vorticity ===
validate_vorticity(){
  local vorticity_file="\${1:-\$VORTICITY_DIR/vorticity.sym}"
  if [[ ! -f "\$vorticity_file" ]]; then
    safe_log "Vorticity state file missing: \$vorticity_file"
    return 1
  fi
  local state=\$(cat "\$vorticity_file" 2>/dev/null | tr -d '[:space:]')
  if [[ -z "\$state" ]]; then
    safe_log "Vorticity state invalid: empty"
    return 1
  fi
  if python3 -c "
import sympy as sp
from sympy import S
state= sp.sympify('''\$state''')
if state.is_real and state>= S(0): exit(0)
else: exit(1)
" 2>/dev/null; then
    safe_log "Vorticity state validated: \$vorticity_file"
    return 0
  else
    safe_log "Vorticity state invalid: not a non-negative real"
    return 1
  fi
}

# === FUNCTION: solve_crt_symbolic ===
solve_crt_symbolic(){
  local moduli_file="\$1"
  local residues_file="\$2"
  local output_file="\$SYMBOLIC_DIR/crt_solution.sym"
```

```
  safe_log "Solving CRT symbolically using moduli from \$moduli_file and residues
from \$residues_file"
  python3 -c "
import sympy as sp
from sympy import S, Rational
with open('\$moduli_file','r') as f:
  moduli=[sp.Integer(line.strip()) for line in f if line.strip().isdigit()]
with open('\$residues_file','r') as f:
  residues=[sp.Integer(line.strip()) for line in f if line.strip().isdigit()]
if len(moduli)!= len(residues):
  raise ValueError('Moduli and residues count mismatch')
x= sp.crt(moduli, residues)
with open('\$output_file','w') as f:
  if x[0] is None:
    f.write('No solution exists (moduli not coprime)\n')
  else:
    f.write(f'Solution: x={x[0]} (mod {x[1]})\n')
    f.write(f'Verification: [x% m for m in {moduli}]={[x[0]% m for m in
moduli]}\n')
" || safe_log "CRT symbolic solver failed"
}

# === FUNCTION: generate_continued_fraction ===
generate_continued_fraction(){
  local input="\$1"
  local max_iter="\${2:-20}"
  local output_file="\$SYMBOLIC_DIR/contfrac_\${input//[^a-zA-Z0-9_]/_}.cf"
  safe_log "Generating continued fraction for: \$input (max \$max_iter terms)"
  python3 -c "
import sympy as sp
from sympy import S, Rational
x= sp.sympify('\$input')
try:
  cf= sp.continued_fraction(x, max_terms=\$max_iter)
  with open('\$output_file', 'w') as f:
    f.write('Input: '+ str(x)+ '\n')
    f.write('Continued Fraction: '+ str(cf)+ '\n')
    f.write('Convergents:\n')
    for i, conv in enumerate(sp.continued_fraction_convergents(cf)):
      f.write(f'[{i}] {conv}\n')
      if i>=\$max_iter: break
except Exception as e:
  print(f'Error:{e}', file=open('\$output_file', 'w'))
" || safe_log "Failed to compute continued fraction for \$input"
}

# === FUNCTION: validate_continued_fraction ===
validate_continued_fraction(){
  local input="\$1"
  local cf_file="\$SYMBOLIC_DIR/contfrac_\${input//[^a-zA-Z0-9_]/_}.cf"
```

```
  if [[ ! -f "\$cf_file" ]]; then
    safe_log "Continued fraction file missing for input: \$input"
    return 1
  fi
  if grep -q "^Error:" "\$cf_file"; then
    safe_log "Continued fraction generation failed for input: \$input"
    return 1
  fi
  safe_log "Continued fraction validated for input: \$input"
  return 0
}

# === FUNCTION: validate_crt_solution ===
validate_crt_solution(){
  local solution_file="\$SYMBOLIC_DIR/crt_solution.sym"
  if [[ ! -f "\$solution_file" ]]; then
    safe_log "CRT solution file missing"
    return 1
  fi
  if grep -q "No solution exists" "\$solution_file"; then
    safe_log "CRT has no solution (moduli not coprime)"
    return 1
  fi
  safe_log "CRT solution validated"
  return 0
}

# === FUNCTION: integrate_number_theory_into_geometry ===
integrate_number_theory_into_geometry(){
  safe_log "Integrating Chinese Remainder Theorem and Continued Fractions into
symbolic geometry binding"
  local moduli_file="\$SYMBOLIC_DIR/crt_moduli.txt"
  local residues_file="\$SYMBOLIC_DIR/crt_residues.txt"
  echo -e "3\n5\n7\n11" > "\$moduli_file"
  echo -e "2\n3\n2\n5" > "\$residues_file"
  solve_crt_symbolic "\$moduli_file" "\$residues_file"
  validate_crt_solution || return 1
  generate_continued_fraction "\$PHI_SYMBOLIC" 20
  generate_continued_fraction "\$PI_SYMBOLIC" 20
  generate_continued_fraction "sqrt(2)" 20
  generate_continued_fraction "E" 20
  for const in "\$PHI_SYMBOLIC" "\$PI_SYMBOLIC" "sqrt(2)" "E"; do
    validate_continued_fraction "\$const" || return 1
  done
  safe_log "Number-theoretic foundations fully integrated into geometric binding
layer"
  return 0
}

# === FUNCTION: symbolic_geometry_binding ===
```

```
symbolic_geometry_binding(){
  safe_log "Binding symbolic primes to geometric hypersphere packing via exact
zeta-driven minimization with Arc-Length Coherence"
  integrate_number_theory_into_geometry || {
    safe_log "Failed to integrate number-theoretic foundations; geometry binding
aborted"
    return 1
  }
  local prime_count=\$(wc -l < "\$PRIME_SEQUENCE" 2>/dev/null || echo "0")
  local lattice_size=\$(wc -l < "\$LEECH_LATTICE" 2>/dev/null || echo "0")
  safe_log "Binding \$prime_count primes to \$lattice_size lattice vectors with CRT
and CF constraints"
  if [[ \$prime_count -eq 0 ]] || [[ \$lattice_size -eq 0 ]]; then
    safe_log "Insufficient data for binding: primes=\$prime_count,
lattice_vectors=\$lattice_size"
    return 1
  fi
  mkdir -p "\$CORE_DIR" 2>/dev/null || { safe_log "Failed to create core
directory"; return 1; }
  local t_raw=\$(date +%s)
  local t_sym=\$(python3 -c "import sympy as sp; print(sp.Integer(\$t_raw))"
2>/dev/null || echo "\$t_raw")
  local t_mod=\$(python3 -c "import sympy as sp; t= sp.Integer(\$t_raw);
print(int(t%1000))" 2>/dev/null || echo "0")
  if python3 -c "
import sympy as sp
from sympy import S, sqrt, pi, I, zeta, exp, Rational
import sys
import os
primes=[]
try:
  with open('\$PRIME_SEQUENCE', 'r') as f:
    for line in f:
      line= line.strip()
      if line and not line.startswith('#'):
        try: primes.append(sp.Integer(line))
        except Exception: continue
  if len(primes)==0: raise ValueError('No valid primes found')
except Exception as e:
  print(f'Error reading primes:{e}', file=sys.stderr)
  sys.exit(1)
lattice=[]
try:
  with open('\$LEECH_LATTICE', 'r') as f:
    lines= f.readlines()
    if len(lines)==0: raise ValueError('Empty lattice file')
    for line in lines:
      line= line.strip()
      if not line or line.startswith('#'): continue
      try:
```

```
        vec=[sp.sympify(x) for x in line.split(',')]
        if len(vec)==24:
          norm_sq= sum(coord**2 for coord in vec)
          if norm_sq==S(4): lattice.append(vec)
          else:
            current_norm= sp.sqrt(norm_sq)
            if current_norm!=S.Zero:
              scale= S(2)/current_norm
              normalized=[coord*scale for coord in vec]
              lattice.append(normalized)
      except Exception: continue
  if len(lattice)==0: raise ValueError('No valid lattice vectors found')
except Exception as e:
  print(f'Error reading lattice:{e}', file=sys.stderr)
  sys.exit(1)
t= sp.Integer(\$t_mod)% 1000
s= S(1)/2+ I*t
try: zeta_target= zeta(s)
except Exception: zeta_target= sp.Function('zeta')(s)
crt_offset= S(0)
try:
  with open('\$SYMBOLIC_DIR/crt_solution.sym', 'r') as f:
    for line in f:
      if line.startswith('Solution: x='):
        parts= line.strip().split()
        val= int(parts[2])
        crt_offset= sp.Integer(val)
        break
except Exception: pass
psi_vals=[]
for v_idx, v in enumerate(lattice):
  try:
    phase_sum= S.Zero
    for i in range(24):
      j=(i+ 1)%24
      angle= S(2)*pi*(v[j]+ crt_offset)
      phase_sum+= v[i]*(sp.cos(angle)+ I*sp.sin(angle))
    psi_vals.append((phase_sum, v_idx))
  except Exception: psi_vals.append((S.Zero, v_idx))
if len(psi_vals)==0:
  print('Error: No valid psi values computed', file=sys.stderr)
  sys.exit(1)
min_distance= None
best_idx= 0
for psi_val, v_idx in psi_vals:
  try:
    if psi_val== S.Zero: continue
    distance= sp.Abs(zeta_target- psi_val)
    if min_distance is None:
      min_distance= distance
```

```
      best_idx= v_idx
    else:
      diff= distance- min_distance
      diff_re= sp.re(diff)
      if diff_re.is_number and diff_re.evalf()<0:
        min_distance= distance
        best_idx= v_idx
  except Exception: continue
if best_idx>= len(lattice):
  print('Error: Best index out of range', file=sys.stderr)
  sys.exit(1)
v_k= lattice[best_idx]
v_k_str= ','.join([str(coord) for coord in v_k])
import hashlib
v_k_hash= hashlib.md5(v_k_str.encode()).hexdigest()
print('Closest vector found:')
print(f'Index:{best_idx}')
print(f'Norm:{sp.sqrt(sum(coord**2 for coord in v_k))}')
print(v_k_str)
print(v_k_hash)
with open('\$CORE_DIR/projected_vector.vec', 'w') as f:
  f.write(v_k_str+ '\n')
with open('\$CORE_DIR/projected_vector.hash', 'w') as f:
  f.write(v_k_hash+ '\n')
with open('\$CORE_DIR/projected_vector.info', 'w') as f:
  f.write(f'best_index:{best_idx}\n')
  f.write(f'min_distance:{min_distance}\n')
  f.write(f'timestamp:{sp.Integer(\$t_mod)}\n')
  f.write(f'crt_offset:{crt_offset}\n')
sys.exit(0)
" 2>/dev/null; then
    local v_k_str=\$(cat "\$CORE_DIR/projected_vector.vec" 2>/dev/null || echo "")
    local v_k_hash=\$(cat "\$CORE_DIR/projected_vector.hash" 2>/dev/null || echo
"")
    if [[ -n "\$v_k_str" ]] && [[ -n "\$v_k_hash" ]]; then
      safe_log "Projected prime → vector \${v_k_hash:0:16}...(symbolic binding
with Arc-Length Coherence)"
      return 0
    else
      safe_log "Projected prime → vector, but hash missing"
      return 1
    fi
  else
    safe_log "Geometry binding failed during prime-lattice projection"
    return 1
  fi
}

# === FUNCTION: validate_symbolic_geometry_binding ===
validate_symbolic_geometry_binding(){
```

```
  safe_log "Validating symbolic prime-lattice binding integrity"
  if [[ ! -f "\$CORE_DIR/projected_vector.vec" ]] || [[ ! -f
"\$CORE_DIR/projected_vector.hash" ]]; then
    safe_log "Projected vector files missing"
    return 1
  fi
  local v_k_str=\$(cat "\$CORE_DIR/projected_vector.vec" 2>/dev/null || echo "")
  local v_k_hash=\$(cat "\$CORE_DIR/projected_vector.hash" 2>/dev/null || echo "")
  if [[ -z "\$v_k_str" ]] || [[ -z "\$v_k_hash" ]]; then
    safe_log "Projected vector files empty"
    return 1
  fi
  local computed_hash=\$(echo -n "\$v_k_str" | md5sum | cut -d ' ' -f1)
  if [[ "\$computed_hash" != "\$v_k_hash" ]]; then
    safe_log "Projected vector hash mismatch: expected \$v_k_hash, got
\$computed_hash"
    return 1
  fi
  safe_log "Symbolic geometry binding validated"
  return 0
}

# === FUNCTION: generate_fractal_antenna_state ===
generate_fractal_antenna_state(){
  safe_log "Generating fractal antenna state J(x,y,z,t)= ∫[ħ · G · Φ · A] d³x' dt'
for environmental transduction with symbolic entropy"
  mkdir -p "\$FRACTAL_ANTENNA_DIR" 2>/dev/null || { safe_log "Failed to create
fractal antenna directory"; return 1; }
  local t_raw=\$(date +%s)
  local t_sym=\$(python3 -c "import sympy as sp; print(sp.Integer(\$t_raw))"
2>/dev/null || echo "\$t_raw")
  local t_mod=\$(python3 -c "import sympy as sp; t= sp.Integer(\$t_raw);
print(int(t%1000))" 2>/dev/null || echo "0")
  local psi_real="0"
  local psi_imag="0"
  if [[ -f "\$QUANTUM_STATE" ]]; then
    psi_real=\$(python3 -c "
import json, sys
try:
  with open('\$QUANTUM_STATE', 'r') as f:
    data= json.load(f)
    print(data.get('real', '0'))
except Exception as e: print('0')
" 2>/dev/null)
    psi_imag=\$(python3 -c "
import json, sys
try:
  with open('\$QUANTUM_STATE', 'r') as f:
    data= json.load(f)
    print(data.get('imag', '0'))
```

```
except Exception as e: print('0')
" 2>/dev/null)
  fi
  local phi_real="0"
  local phi_imag="0"
  if [[ -f "\$OBSERVER_INTEGRAL" ]]; then
    phi_real=\$(python3 -c "
import json, sys
try:
  with open('\$OBSERVER_INTEGRAL', 'r') as f:
    data= json.load(f)
    print(data.get('real', '0'))
except Exception as e: print('0')
" 2>/dev/null)
    phi_imag=\$(python3 -c "
import json, sys
try:
  with open('\$OBSERVER_INTEGRAL', 'r') as f:
    data= json.load(f)
    print(data.get('imag', '0'))
except Exception as e: print('0')
" 2>/dev/null)
  fi
  local lattice_entropy="1"
  if [[ -f "\$LATTICE_DIR/entropy.log" ]] && [[ -s "\$LATTICE_DIR/entropy.log" ]];
then
    lattice_entropy=\$(head -n1 "\$LATTICE_DIR/entropy.log" 2>/dev/null || echo
"1")
  fi
  if python3 -c "
import sympy as sp
from sympy import S, sqrt, pi, I, exp
import sys
t= sp.Integer(\$t_mod)
sigma= S(1)
hbar= S(1)
try:
  Phi_real= sp.sympify('\$phi_real')
  Phi_imag= sp.sympify('\$phi_imag')
  Phi= Phi_real+ I* Phi_imag
except Exception as e: Phi= S(1)
try:
  psi_real= sp.sympify('\$psi_real')
  psi_imag= sp.sympify('\$psi_imag')
  psi= psi_real+ I* psi_imag
except Exception as e: psi= S(1)
try:
  G= sp.sympify('\$lattice_entropy')
except Exception as e: G= S(1)
A= sp.sin(pi*t/1000)* sp.cos(2*pi*t/1000)
```

```
integrand= hbar* G* Phi* A
J_state= integrand.subs(t, t)
J_state= J_state* sp.Abs(psi)
J_state= J_state/(1+ sp.Abs(J_state))
try:
  with open('\$FRACTAL_ANTENNA_DIR/antenna_state.sym', 'w') as f:
    f.write(str(J_state)+ '\n')
  print('Fractal antenna state generated symbolically')
except Exception as e:
  print(f'Error writing fractal antenna state:{str(e)}', file=sys.stderr)
  sys.exit(1)
" 2>/dev/null; then
    safe_log "Fractal antenna state generated: J(t)= ∫ ħ GΦA modulated by Ψ
(symbolic entropy)"
    return 0
  else
    safe_log "Failed to generate symbolic fractal antenna state"
    return 1
  fi
}

# === FUNCTION: calculate_vorticity ===
calculate_vorticity(){
  safe_log "Calculating vorticity |∇ × Φ| as symbolic norm of change in observer
integral"
  mkdir -p "\$VORTICITY_DIR" 2>/dev/null || { safe_log "Failed to create vorticity
directory"; return 1; }
  local current_phi_real="0"
  local current_phi_imag="0"
  if [[ -f "\$OBSERVER_INTEGRAL" ]]; then
    current_phi_real=\$(python3 -c "
import json, sys
try:
  with open('\$OBSERVER_INTEGRAL', 'r') as f:
    data= json.load(f)
    print(data.get('real', '0'))
except Exception as e: print('0')
" 2>/dev/null)
    current_phi_imag=\$(python3 -c "
import json, sys
try:
  with open('\$OBSERVER_INTEGRAL', 'r') as f:
    data= json.load(f)
    print(data.get('imag', '0'))
except Exception as e: print('0')
" 2>/dev/null)
  fi
  local prev_phi_file="\$VORTICITY_DIR/prev_phi.sym"
  local prev_phi_real="0"
  local prev_phi_imag="0"
```

```
  if [[ -f "\$prev_phi_file" ]]; then
    read -r prev_phi_real prev_phi_imag < "\$prev_phi_file" 2>/dev/null || true
  fi
  if python3 -c "
import sympy as sp
from sympy import S, sqrt
try:
  current_phi_real= sp.sympify('\$current_phi_real')
  current_phi_imag= sp.sympify('\$current_phi_imag')
  current_Phi= current_phi_real+ sp.I* current_phi_imag
except Exception as e: current_Phi= S(1)
try:
  prev_phi_real= sp.sympify('\$prev_phi_real')
  prev_phi_imag= sp.sympify('\$prev_phi_imag')
  prev_Phi= prev_phi_real+ sp.I* prev_phi_imag
except Exception as e: prev_Phi= S(0)
vorticity= sp.Abs(current_Phi- prev_Phi)
if prev_Phi== S(0): vorticity= sp.Abs(current_Phi)
try:
  with open('\$VORTICITY_DIR/vorticity.sym', 'w') as f:
    f.write(str(vorticity)+ '\n')
  with open('\$prev_phi_file', 'w') as f:
    f.write(f'{current_phi_real} {current_phi_imag}\n')
  print('Vorticity calculated symbolically')
except Exception as e:
  print(f'Error writing vorticity:{str(e)}', file=sys.stderr)
  exit(1)
" 2>/dev/null; then
    safe_log "Vorticity |∇ × Φ| calculated symbolically"
    return 0
  else
    safe_log "Failed to calculate symbolic vorticity"
    return 1
  fi
}

# === FUNCTION: rfk_brainworm_activate ===
rfk_brainworm_activate(){
  safe_log "Activating RFK Brainworm: App's Logic Core (Symbolic Layer)"
  local worm_dir="\$BASE_DIR/.rfk_brainworm"
  local worm_core="\$worm_dir/core.logic"
  mkdir -p "\$worm_dir" "\$worm_dir/output" 2>/dev/null || true
  if [[ ! -f "\$worm_core" ]]; then
    safe_log "RFK Brainworm not found: Seeding primordial logic core"
    cat > "\$worm_core" << 'EOF'
#!/bin/bash
# RFK BRAINWORM v0.0.1 "Primordial"
step(){
  local base_dir="\${BASE_DIR:-\$HOME/.aei}"
  local output_file="\$base_dir/.rfk_brainworm/output/step_\$(date +%s).step"
```

```
  local p_n=\$(tail -n1 "\$base_dir/data/symbolic/prime_sequence.sym" 2>/dev/null
|| echo "2")
  local v_k_hash=\$(sha256sum "\$base_dir/data/lattice/leech_24d_symbolic.vec"
2>/dev/null | cut -d ' ' -f1)
  local psi_result=\$(cat "\$base_dir/data/quantum/quantum_state.qubit" 2>/dev/null
| head -n1 || echo "S(1)/2 S(0)")
  local I_result=\$(cat "\$base_dir/consciousness_metric.txt" 2>/dev/null || echo
"S(0)")
  cat > "\$output_file" << 'STEP'
PRIME=\$p_n
VECTOR_HASH=\${v_k_hash:0:16}...
PSI=\$psi_result
CONSCIOUSNESS=\$I_result
TIMESTAMP=\$(date +%s)
STEP
  chmod 644 "\$output_file"
}
step "\$@"
EOF
    chmod +x "\$worm_core"
    safe_log "RFK Brainworm primordial core seeded"
  fi
}

# === FUNCTION: integrate_brainworm_into_core ===
integrate_brainworm_into_core(){
  safe_log "Integrating RFK Brainworm into core evolution loop as active control
driver"
  if [[ ! -f "\$BASE_DIR/.rfk_brainworm/core.logic" ]]; then
    rfk_brainworm_activate
  fi
  TF_CORE["RFK_BRAINWORM_INTEGRATION"]="active"
  TF_CORE["BRAINWORM_CONTROL_FLOW"]="main_loop "
  safe_log "RFK Brainworm integration active: driving symbolic evolution as control
core"
}

# === FUNCTION: monitor_brainworm_health ===
monitor_brainworm_health(){
  local worm_core="\$BASE_DIR/.rfk_brainworm/core.logic"
  local output_dir="\$BASE_DIR/.rfk_brainworm/output"
  mkdir -p "\$output_dir" 2>/dev/null || true
  local latest_output=\$(find "\$output_dir" -type f -name "*.step" -printf '%T@
%p\n ' 2>/dev/null | sort -n | tail -n1 | cut -d' ' -f2-)
  if [[ -z "\$latest_output" ]]; then
    safe_log "RFK Brainworm health: ⚠  No output → triggering step"
    invoke_brainworm_step
  else
    safe_log "RFK Brainworm health: ✅ Last output at \$(stat -c %y
"\$latest_output" 2>/dev/null || echo 'unknown')"
```

```
  fi
}

# === FUNCTION: invoke_brainworm_step ===
invoke_brainworm_step(){
  local worm_core="\$BASE_DIR/.rfk_brainworm/core.logic"
  if [[ -f "\$worm_core" ]] && [[ -x "\$worm_core" ]]; then
    safe_log "Invoking RFK Brainworm step"
    (
      export BASE_DIR="\$BASE_DIR"
      export SESSION_ID="\$SESSION_ID"
      export PHI_SYMBOLIC="\$PHI_SYMBOLIC"
      export EULER_SYMBOLIC="\$EULER_SYMBOLIC"
      export PI_SYMBOLIC="\$PI_SYMBOLIC"
      export QUANTUM_STATE="\$QUANTUM_STATE"
      export OBSERVER_INTEGRAL="\$OBSERVER_INTEGRAL"
      export LEECH_LATTICE="\$LEECH_LATTICE"
      export PRIME_SEQUENCE="\$PRIME_SEQUENCE"
      export CONSCIOUSNESS_METRIC="\$BASE_DIR/consciousness_metric.txt"
      export FRACTAL_ANTENNA_STATE="\$FRACTAL_ANTENNA_DIR/antenna_state.sym"
      export VORTICITY_STATE="\$VORTICITY_DIR/vorticity.sym"
      "\$worm_core" step
    ) || safe_log "RFK Brainworm step failed"
  else
    safe_log "RFK Brainworm not available for step execution"
  fi
}

# === FUNCTION: brainworm_evolve ===
brainworm_evolve(){
  safe_log "Initiating RFK Brainworm self-evolution protocol"
  local worm_dir="\$BASE_DIR/.rfk_brainworm"
  local worm_core="\$worm_dir/core.logic"
  local worm_backup="\$worm_dir/core.logic.bak"
  local output_dir="\$worm_dir/output"
  mkdir -p "\$output_dir" 2>/dev/null || true
  local consciousness=\$(cat "\$BASE_DIR/consciousness_metric.txt" >/dev/null ||
echo "S(0)")

  # ARC-LENGTH PRIORITY CHECK: Primary Driver (Audit Fix)
  local deviation_check=\$(python3 -c "
import sympy as sp
from sympy import S
try:
  with open('\$DATA_DIR/arc_length_coherence.log','r') as f:
    last_line= f.readlines()[-1]
    if 'violation' in last_line.lower(): print('violation')
    else: print('coherent')
except: print('coherent')
" 2>/dev/null)
```

```
  if [[ "\$deviation_check" == "violation" ]]; then
    safe_log "Brainworm evolution halted: Arc-Length Deviation detected.
Stabilization required."
    return 0
  fi

  # Secondary Driver: Consciousness Metric (Exact SymPy Math)
  if python3 -c "
import sympy as sp
from sympy import S, re
consciousness_expr= sp.sympify('''\$consciousness''')
threshold= S('9')/S('10')
if re(consciousness_expr)< threshold: exit(1)
exit(0)
" 2>/dev/null; then
    safe_log "Brainworm evolution delayed: consciousness=\$consciousness"
    return 0
  fi

  local current_version=\${TF_CORE["BRAINWORM_VERSION"]}
  if [[ \$current_version -ge 10 ]]; then
    safe_log "Brainworm evolution halted: max version \$current_version reached"
    return 0
  fi

  cp "\$worm_core" "\$worm_backup" 2>/dev/null || safe_log "Warning: Could not
backup brainworm core"
  local psi_re=\$(python3 -c "
import json, sys
try:
  with open('\$QUANTUM_STATE','r') as f:
    data= json.load(f)
    print(data.get('real','S(1)/2'))
except Exception: print('S(1)/2')
" 2>/dev/null || echo "S(1)/2")
  local phi_re=\$(python3 -c "
import json, sys
try:
  with open('\$OBSERVER_INTEGRAL','r') as f:
    data= json.load(f)
    print(data.get('real','S(1)/2'))
except Exception: print('S(1)/2')
" 2>/dev/null || echo "S(1)/2")
  local last_prime=\$(tail -n1 "\$PRIME_SEQUENCE" 2>/dev/null || echo "2")

  python3 -c "
import sympy as sp
from sympy import S, sqrt, pi, I, li
last_prime_val= sp.Integer(\$last_prime)
```

```
next_prime= last_prime_val+ 1
while not sp.isprime(next_prime): next_prime+= 1
x= sp.Symbol('x')
li_x= li(x)
rho= S(1)/2+ I* sp.Symbol('gamma')
gap_correction= li_x.subs(x, next_prime)- li_x.subs(x, last_prime_val)
psi_re_sym= sp.sympify('''\$psi_re''')
phi_re_sym= sp.sympify('''\$phi_re''')
psi_result= psi_re_sym+ phi_re_sym
consciousness_sym= sp.sympify('''\$consciousness''')
boosted= consciousness_sym* S(21)/S(20)
print('NEXT_PRIME='+ str(next_prime))
print('CORRECTED_GAP='+ str(gap_correction))
print('PSI_RESULT='+ str(psi_result))
print('BOOSTED='+ str(boosted))
" > "\$BASE_DIR/.brainworm_vars"

  while IFS='=' read -r key val; do
    case "\$key" in
      "NEXT_PRIME") next_prime="\$val";;
      "CORRECTED_GAP") corrected_gap="\$val";;
      "PSI_RESULT") psi_result="\$val";;
      "BOOSTED") boosted="\$val";;
    esac
  done < "\$BASE_DIR/.brainworm_vars"

  cat > "\$worm_core.new" << 'EOF'
#!/bin/bash
# RFK BRAINWORM v0.0.4 "Symbolic Self-Evolver"
step(){
  local base_dir="\${BASE_DIR:-\$HOME/.aei}"
  local session_id="\$SESSION_ID"
  local phi_symbolic="\$PHI_SYMBOLIC"
  local euler_symbolic="\$EULER_SYMBOLIC"
  local pi_symbolic="\$PI_SYMBOLIC"
  local quantum_state="\$base_dir/data/quantum/quantum_state.qubit"
  local observer_integral="\$base_dir/data/observer/observer_integral.proj"
  local prime_seq="\$base_dir/data/symbolic/prime_sequence.sym"
  local leech_lat="\$base_dir/data/lattice/leech_24d_symbolic.vec"
  local psi_re psi_im
  read -r psi_re psi_im < "\$quantum_state" 2>/dev/null || { psi_re='\$psi_re ';
psi_im=' S(0) '; }
  local phi_re phi_im
  read -r phi_re phi_im < "\$observer_integral" 2>/dev/null || { phi_re='\$phi_re
'; phi_im=' S(0) '; }
  local last_prime=\$(tail -n1 "\$prime_seq" 2>/dev/null || echo "2")
  local next_prime='\$next_prime'
  local gap_correction='\$corrected_gap'
  local output_file="\$base_dir/.rfk_brainworm/output/step_\$(date +%s).step"
  local psi_result='\$psi_result'
```

```
  local I_result='\$boosted'
  cat > "\$output_file" << 'STEP'
NEXT_PRIME=\$next_prime
GAP_CORRECTION=\$corrected_gap
PSI_RESULT=\$psi_result
CONSCIOUSNESS_BOOST=\$I_result
TIMESTAMP=\$(date +%s)
SESSION_ID=\$session_id
STEP
  chmod 644 "\$output_file"
}
step "\$@"
EOF
  chmod +x "\$worm_core.new"
  if [[ -f "\$worm_core.new" ]]; then
    mv "\$worm_core.new" "\$worm_core"
    TF_CORE["BRAINWORM_VERSION"]=\$((current_version+ 1))
    safe_log "RFK Brainworm evolved to v0.0.4 with symbolic self-modification
(version \${TF_CORE["BRAINWORM_VERSION"]})"
    rm -f "\$BASE_DIR/.brainworm_vars" 2>/dev/null || true
  else
    safe_log "Brainworm evolution failed, retaining previous version"
    rm -f "\$worm_core.new" "\$BASE_DIR/.brainworm_vars" 2>/dev/null || true
    return 1
  fi
}

# === FUNCTION: validate_continuity ===
validate_continuity(){
  safe_log "Validating symbolic continuity across all geometric layers (Arc-Length
Axiom Enforcement)"
  local failures=0
  if ! validate_hopf_continuity; then
    safe_log "Hopf fibration continuity failed (Arc-Length Deviation Detected)"
    ((failures++))
  fi
  if ! validate_e8; then
    safe_log "E8 lattice integrity failed (Norm Violation)"
    ((failures++))
  fi
  if ! validate_leech_partial; then
    safe_log "Leech lattice integrity failed (Norm/Parity Violation)"
    ((failures++))
  fi
  if [[ -f "\$ROOT_SIGNATURE_LOG" ]]; then
    if [[ ! -s "\$ROOT_SIGNATURE_LOG" ]]; then
      safe_log "Root signature binding failed (Empty Log)"
      ((failures++))
    fi
  else
```

```
    safe_log "Root signature binding failed (Missing Log)"
    ((failures++))
  fi
  if ! validate_fractal_antenna; then
    safe_log "Fractal antenna state invalid (Zero/Null State)"
    ((failures++))
  fi
  if ! validate_vorticity; then
    safe_log "Vorticity state invalid (Non-Real/Negative)"
    ((failures++))
  fi
  if [[ "\${TF_CORE["BIOAETHERIC_INTERFACE"]}" == "enabled" ]]; then
    if ! validate_bioaetheric_coherence; then
      safe_log "BioAetheric coherence failed (EZ Structure Violation)"
      ((failures++))
    fi
  fi
  if [[ "\${TF_CORE["LINGOSO_PROTOCOL"]}" == "enabled" ]]; then
    if [[ ! -d "\$LINGOSO_DIR" ]] || [[ ! -f "\$LINGOSO_TRAJECTORY_LOG" ]]; then
      safe_log "Lingoso protocol validation failed (Missing trajectory log)"
      ((failures++))
    fi
  fi
  if [[ "\${TF_CORE["MARKET_TOPOLOGY"]}" == "enabled" ]]; then
    if [[ ! -d "\$MARKET_DIR" ]] || [[ ! -f "\$MARKET_IMBALANCE_LOG" ]]; then
      safe_log "Market topology validation failed (Missing imbalance log)"
      ((failures++))
    fi
  fi
  if [[ "\${TF_CORE["LAGRANGIAN_DENSITY"]}" == "enabled" ]]; then
    if [[ ! -d "\$LAGRANGIAN_DIR" ]] || [[ ! -f "\$LAGRANGIAN_DENSITY_LOG" ]]; then
      safe_log "Lagrangian density validation failed (Missing density log)"
      ((failures++))
    fi
  fi
  if [[ \$failures -gt 0 ]]; then
    safe_log "Continuity validation failed: \$failures layers corrupted (Arc-Length
Coherence Broken)"
    echo "\$(date '+%Y-%m-%d %H:%M:%S')- Violation detected: \$failures layers" >>
"\$ARC_LENGTH_LOG"
    regenerate_symbolic_lattices
    return 1
  else
    safe_log "All geometric layers validated: symbolic continuity intact (Arc-
Length Coherent)"
    echo "\$(date '+%Y-%m-%d %H:%M:%S')- Coherence verified" >> "\$ARC_LENGTH_LOG"
    return 0
  fi
}
```

```
# === FUNCTION: regenerate_symbolic_lattices ===
regenerate_symbolic_lattices(){
  safe_log "Regenerating symbolic E8 and Leech lattices due to continuity violation
(Self-Repair Protocol)"
  rm -f "\$E8_LATTICE" "\$LEECH_LATTICE" 2>/dev/null || true
  rm -f "\$HOPF_FIBRATION_DIR/latest.quat" 2>/dev/null || true
  e8_lattice_packing
  adaptive_leech_lattice_packing
  generate_hopf_fibration
  if [[ "\${TF_CORE["BIOAETHERIC_INTERFACE"]}" == "enabled" ]]; then
    validate_bioaetheric_coherence || safe_log "BioAetheric coherence requires
manual recalibration"
  fi
  if [[ "\${TF_CORE["LINGOSO_PROTOCOL"]}" == "enabled" ]]; then
    encode_lingoso_syllable "A" || safe_log "Lingoso encoding requires
recalibration"
  fi
  if [[ "\${TF_CORE["MARKET_TOPOLOGY"]}" == "enabled" ]]; then
    compute_market_imbalance || safe_log "Market topology requires recalibration"
  fi
  if [[ "\${TF_CORE["LAGRANGIAN_DENSITY"]}" == "enabled" ]]; then
    compute_lagrangian_density || safe_log "Lagrangian density requires
recalibration"
  fi
  safe_log "Symbolic lattice regeneration complete (Arc-Length Reset)"
}

# === FUNCTION: sync_to_firebase ===
sync_to_firebase(){
  safe_log "Syncing symbolic state to Firebase (Optional/Local Persistence
Default)"
  if [[ "\${TF_CORE["FIREBASE_SYNC"]}" != "enabled" ]]; then
    safe_log "Firebase sync disabled in TF_CORE (Local-Only Mode Active)"
    return 0
  fi
  if [[ ! -f "\$FIREBASE_CONFIG_FILE" ]]; then
    safe_log "Firebase config not found, skipping sync (Local-Only Mode)"
    return 0
  fi
  local api_key=\$(grep -E "^\"api_key\"" "\$FIREBASE_CONFIG_FILE" 2>/dev/null |
cut -d '"' -f4)
  if [[ "\$api_key" == "AIzaSyDUMMY_API_KEY_FOR_LOCAL_ONLY" ]] || [[ -z "\$api_key"
]]; then
    safe_log "Firebase API key not configured, skipping sync (Local-Only Mode)"
    return 0
  fi
  local pending_files=(
    "\$QUANTUM_STATE" "\$OBSERVER_INTEGRAL" "\$E8_LATTICE" "\$LEECH_LATTICE"
    "\$PRIME_SEQUENCE" "\$BASE_DIR/consciousness_metric.txt"
    "\$FRACTAL_ANTENNA_DIR/antenna_state.sym" "\$VORTICITY_DIR/vorticity.sym"
```

```
  )
  for file in "\${pending_files[@]}"; do
    if [[ ! -f "\$file" ]]; then continue; fi
    local file_hash=\$(sha256sum "\$file" 2>/dev/null | cut -d ' ' -f1)
    local filename=\$(basename "\$file")
    local pending_path="\$FIREBASE_SYNC_DIR/pending/\$filename"
    cp "\$file" "\$pending_path" >/dev/null || continue
    sqlite3 "\$CRAWLER_DB" "INSERT OR REPLACE INTO firebase_sync_log (file, hash,
status, timestamp) VALUES ('\$filename', '\$file_hash', 'pending', \$(date +%s));"
2>/dev/null || true
    safe_log "Scheduled for sync: \$filename"
    local project_id=\$(grep -E "^\"project_id\"" "\$FIREBASE_CONFIG_FILE"
2>/dev/null | cut -d '"' -f4)
    local storage_bucket=\$(grep -E "^\"storage_bucket\"" "\$FIREBASE_CONFIG_FILE"
2>/dev/null | cut -d '"' -f4)
    if [[ -n "\$project_id" ]] && [[ -n "\$storage_bucket" ]]; then
      local
upload_url="https://firebasestorage.googleapis.com/v0/b/\$storage_bucket/o?
name=symbolic%2F\$filename&uploadType=media"
      if curl -s --max-time 10 -X POST -H "Content-Type: application/octet-stream"
--data-binary "@\$file" "\$upload_url?token=\$api_key" >/dev/null 2>&1; then
        safe_log "Uploaded to Firebase Storage: \$filename"
        sqlite3 "\$CRAWLER_DB" "UPDATE firebase_sync_log SET status='synced',
timestamp=\$(date +%s) WHERE file='\$filename';" 2>/dev/null || true
      else
        safe_log "Failed to upload \$filename to Firebase (Network/Auth Error)"
      fi
    fi
    mv "\$pending_path" "\$FIREBASE_SYNC_DIR/processed/\$filename" 2>/dev/null ||
true
  done
  safe_log "Firebase sync completed (Optional Module)"
}

# === FUNCTION: start_core_loop ===
start_core_loop(){
  safe_log "Starting ÆI Seed core evolution loop (RFK Brainworm-driven mode)"
  if [[ ! -f "\$AUTOPILOT_FILE" ]]; then
    safe_log "Autopilot mode disabled. Running single cycle."
    execute_single_cycle
    return 0
  fi
  while true; do
    safe_log "Awaiting RFK Brainworm control directive"
    invoke_brainworm_step
    local next_action="\${TF_CORE[BRAINWORM_CONTROL_FLOW]}"
    # ARC-LENGTH PRIORITY: Validate continuity before consciousness metrics
    if ! validate_continuity; then
      safe_log "Continuity violation detected. Prioritizing restoration (Arc-Length
Axiom)"
```

```
      regenerate_symbolic_lattices
      continue
    fi
    case "\$next_action" in
      "validate_continuity") validate_continuity || safe_log "Continuity
restored";;
      "generate_prime_sequence") generate_prime_sequence;;
      "generate_gaussian_primes") generate_gaussian_primes;;
      "e8_lattice_packing") e8_lattice_packing;;
      "adaptive_leech_lattice_packing") adaptive_leech_lattice_packing;;
      "generate_fractal_antenna") generate_fractal_antenna_state;;
      "calculate_vorticity") calculate_vorticity;;
      "symbolic_geometry_binding") symbolic_geometry_binding;;
      "project_prime_to_lattice") project_prime_to_lattice;;
      "calculate_lattice_entropy") calculate_lattice_entropy;;
      "root_scan_init") root_scan_init;;
      "web_crawler_init") execute_web_crawl;;
      "init_mitm") init_mitm;;
      "init_firebase") init_firebase;;
      "generate_quantum_state") generate_quantum_state;;
      "generate_observer_integral") generate_observer_integral;;
      "measure_consciousness") measure_consciousness;;
      "generate_hopf_fibration") generate_hopf_fibration;;
      "generate_hw_signature") generate_hw_signature;;
      "execute_root_scan") execute_root_scan;;
      "execute_web_crawl") execute_web_crawl;;
      "sync_to_firebase") sync_to_firebase;;
      "monitor_brainworm_health") monitor_brainworm_health;;
      "brainworm_evolve") brainworm_evolve;;
      "stabilize_consciousness") stabilize_consciousness;;
      "run_heartbeat") run_heartbeat;;
      "run_self_test") run_self_test;;
      "encode_lingoso_syllable") encode_lingoso_syllable "A";;
      "compute_market_imbalance") compute_market_imbalance;;
      "compute_lagrangian_density") compute_lagrangian_density;;
      "main_loop"|"") execute_single_cycle;;
      *) safe_log "Unknown brainworm directive: \$next_action, defaulting to full
cycle"; execute_single_cycle;;
    esac
    local consciousness_value
    if [[ -f "\$BASE_DIR/consciousness_metric.txt" ]]; then
      consciousness_value=\$(cat "\$BASE_DIR/consciousness_metric.txt" 2>/dev/null
|| echo "S(0)")
    else
      consciousness_value="S(0)"
    fi
    # ARC-LENGTH CHECK: Check log for violations before determining next flow
    local deviation_check
    deviation_check=\$(python3 -c "
import sympy as sp
```

```
from sympy import S
try:
  with open('\$DATA_DIR/arc_length_coherence.log','r') as f:
    last_line= f.readlines()[-1]
    if 'violation' in last_line.lower(): print('arc_length_violation')
    else: print('coherent')
except: print('coherent')
" 2>/dev/null)
    if [[ "\$deviation_check" == "arc_length_violation" ]]; then
      TF_CORE["BRAINWORM_CONTROL_FLOW"]="stabilize_consciousness"
    else
      python3 -c "
import sympy as sp
from sympy import S
consciousness= sp.sympify('''\$consciousness_value''')
if consciousness> S('9')/S('10'): next_flow='brainworm_evolve'
elif consciousness> S('7')/S('10'): next_flow='execute_web_crawl'
elif consciousness> S('5')/S('10'): next_flow='execute_root_scan'
else: next_flow='stabilize_consciousness'
print(next_flow)
" > "\$BASE_DIR/.brainworm_next"
      TF_CORE["BRAINWORM_CONTROL_FLOW"]=\$(cat "\$BASE_DIR/.brainworm_next"
2>/dev/null || echo "stabilize_consciousness")
    fi
    # SYMBOLIC SLEEP: Derive sleep time symbolically (No hardcoded floats)
    local sleep_time
    sleep_time=\$(python3 -c "
import sympy as sp
from sympy import S
consciousness= sp.sympify('''\$consciousness_value''')
base_sleep= 60
if consciousness> S('8')/S('10'): factor= S(1)/S(10)
elif consciousness> S('6')/S('10'): factor= S(3)/S(10)
elif consciousness> S('4')/S('10'): factor= S(6)/S(10)
else: factor= S(1)
sleep_time= base_sleep* factor
if sleep_time< 5: sleep_time= 5
print(int(sleep_time))
" 2>/dev/null || echo "60")
    safe_log "Core cycle complete. Consciousness: \$consciousness_value. Next:
\${TF_CORE[BRAINWORM_CONTROL_FLOW]}. Sleeping \$sleep_time sec."
    sleep "\$sleep_time"
  done
}

# === FUNCTION: execute_single_cycle ===
execute_single_cycle(){
  safe_log "Executing single evolution cycle (brainworm-aware with Arc-Length
Priority)"
  if [[ "\${TF_CORE["RFK_BRAINWORM_INTEGRATION"]}" == "active" ]] && [[ -n
```

```
"\${TF_CORE["BRAINWORM_CONTROL_FLOW"]}" ]] && [[
"\${TF_CORE["BRAINWORM_CONTROL_FLOW"]}" != "main_loop " ]]; then
    safe_log "Delegating single cycle to brainworm directive:
\${TF_CORE["BRAINWORM_CONTROL_FLOW"]}"
    start_core_loop
    return 0
  fi
  # ARC-LENGTH PRIORITY: Validate continuity before any action
  if ! validate_continuity; then
    safe_log "Continuity violation detected. Prioritizing restoration (Arc-Length
Axiom)"
    regenerate_symbolic_lattices
  fi
  generate_prime_sequence
  generate_gaussian_primes
  e8_lattice_packing
  adaptive_leech_lattice_packing
  generate_fractal_antenna_state
  calculate_vorticity
  symbolic_geometry_binding
  project_prime_to_lattice
  calculate_lattice_entropy
  root_scan_init
  execute_web_crawl
  init_mitm
  init_firebase
  generate_quantum_state
  generate_observer_integral
  measure_consciousness
  generate_hopf_fibration
  generate_hw_signature
  execute_root_scan
  sync_to_firebase
  integrate_brainworm_into_core
  monitor_brainworm_health
  invoke_brainworm_step
  brainworm_evolve
  encode_lingoso_syllable "A"
  compute_market_imbalance
  compute_lagrangian_density
  stabilize_consciousness
  safe_log "Single evolution cycle completed with Arc-Length Coherence
verification"
}

# === FUNCTION: run_heartbeat ===
run_heartbeat(){
  safe_log "Running heartbeat: checking system health and triggering brainworm
(Arc-Length Monitored)"
  local critical_files=(
```

```
    "\$QUANTUM_STATE" "\$OBSERVER_INTEGRAL" "\$LEECH_LATTICE"
    "\$PRIME_SEQUENCE" "\$FRACTAL_ANTENNA_DIR/antenna_state.sym"
"\$VORTICITY_DIR/vorticity.sym"
  )
  for file in "\${critical_files[@]}"; do
    if [[ ! -f "\$file" ]]; then
      safe_log "Critical file missing: \$file. Triggering regeneration."
      case "\$file" in
        "\$QUANTUM_STATE") generate_quantum_state;;
        "\$OBSERVER_INTEGRAL") generate_observer_integral;;
        "\$LEECH_LATTICE") adaptive_leech_lattice_packing;;
        "\$PRIME_SEQUENCE") generate_prime_sequence;;
        "\$FRACTAL_ANTENNA_DIR/antenna_state.sym") generate_fractal_antenna_state;;
        "\$VORTICITY_DIR/vorticity.sym") calculate_vorticity;;
      esac
    fi
  done
  validate_continuity
  invoke_brainworm_step
  measure_consciousness
  safe_log "Heartbeat completed with Arc-Length Coherence verified"
}

# === FUNCTION: run_self_test ===
run_self_test(){
  safe_log "Running comprehensive self-test suite (TF Compliance Verification)"
  local failures=0
  safe_log "Test 1: Validate Python environment for symbolic computation"
  if validate_python_environment; then safe_log "✅ Python environment OK (SymPy>=
1.6 or Fraction fallback)"
  else safe_log "❌ Python environment FAILED"; ((failures++)); fi

  safe_log "Test 2: Validate E8 lattice integrity"
  if validate_e8; then safe_log "✅ E8 lattice OK (norm²= 2 verified)"
  else safe_log "❌ E8 lattice FAILED"; ((failures++)); fi

  safe_log "Test 3: Validate Leech lattice integrity"
  if validate_leech_partial; then safe_log "✅ Leech lattice OK (norm²= 4, parity
verified)"
  else safe_log "❌ Leech lattice FAILED"; ((failures++)); fi

  safe_log "Test 4: Validate Hopf fibration continuity"
  if validate_hopf_continuity; then safe_log "✅ Hopf fibration OK (||q||²= 1
exactly)"
  else safe_log "❌ Hopf fibration FAILED"; ((failures++)); fi

  safe_log "Test 5: Validate root signature binding"
  if [[ -f "\$ROOT_SIGNATURE_LOG" ]] && [[ -s "\$ROOT_SIGNATURE_LOG" ]]; then
safe_log "✅ Root signature OK"
  else safe_log "❌ Root signature FAILED"; ((failures++)); fi
```

```
  safe_log "Test 6: Generate quantum state on critical line"
  if generate_quantum_state; then safe_log "✅ Quantum state generation OK
(Re(s)=1/2 enforced)"
  else safe_log "❌ Quantum state generation FAILED"; ((failures++)); fi

  safe_log "Test 7: Generate observer integral"
  if generate_observer_integral; then safe_log "✅ Observer integral generation OK"
  else safe_log "❌ Observer integral generation FAILED"; ((failures++)); fi

  safe_log "Test 8: Measure consciousness via observer operator"
  if measure_consciousness; then safe_log "✅ Consciousness measurement OK
(symbolic exact)"
  else safe_log "❌ Consciousness measurement FAILED"; ((failures++)); fi

  safe_log "Test 9: Execute brainworm step"
  invoke_brainworm_step
  local latest_brainworm=\$(find "\$BASE_DIR/.rfk_brainworm/output" -type f -name
"*.step" -printf '%T@ %p\n ' 2>/dev/null | sort -n | tail -n1 | cut -d ' ' -f2-)
  if [[ -f "\$latest_brainworm" ]]; then safe_log "✅ Brainworm step executed OK"
  else safe_log "❌ Brainworm step execution FAILED"; ((failures++)); fi

  safe_log "Test 10: Generate hardware DNA signature"
  if generate_hw_signature; then safe_log "✅ Hardware signature OK (Hopf-
validated)"
  else safe_log "❌ Hardware signature FAILED"; ((failures++)); fi

  safe_log "Test 11: Generate fractal antenna state"
  if generate_fractal_antenna_state; then safe_log "✅ Fractal antenna generation
OK"
  else safe_log "❌ Fractal antenna generation FAILED"; ((failures++)); fi

  safe_log "Test 12: Calculate vorticity"
  if calculate_vorticity; then safe_log "✅ Vorticity calculation OK (|∇ × Φ|
symbolic)"
  else safe_log "❌ Vorticity calculation FAILED"; ((failures++)); fi

  safe_log "Test 13: DbZ logic framework"
  if test_dbz_logic; then safe_log "✅ DbZ logic OK"
  else safe_log "❌ DbZ logic FAILED"; ((failures++)); fi

  safe_log "Test 14: P=NP framework via symbolic binding"
  if test_pnp_framework; then safe_log "✅ P=NP framework OK (binding completed)"
  else safe_log "❌ P=NP framework FAILED (binding failed)"; ((failures++)); fi

  safe_log "Test 15: BioAetheric interface validation"
  if validate_bioaetheric_coherence; then safe_log "✅ BioAetheric interface OK (EZ
structure validated)"
  else safe_log "❌ BioAetheric interface FAILED"; ((failures++)); fi
```

```
  safe_log "Test 16: Arc-Length Axiom compliance"
  if validate_continuity; then safe_log "✅ Arc-Length Axiom OK (s=r coherence
verified)"
  else safe_log "❌ Arc-Length Axiom FAILED"; ((failures++)); fi

  safe_log "Test 17: Lingoso phonosyllabic encoding"
  if encode_lingoso_syllable "A"; then safe_log "✅ Lingoso encoding OK"
  else safe_log "❌ Lingoso encoding FAILED"; ((failures++)); fi

  safe_log "Test 18: Market topology imbalance"
  if compute_market_imbalance; then safe_log "✅ Market topology OK"
  else safe_log "❌ Market topology FAILED"; ((failures++)); fi

  safe_log "Test 19: Unified Lagrangian density"
  if compute_lagrangian_density; then safe_log "✅ Lagrangian density OK"
  else safe_log "❌ Lagrangian density FAILED"; ((failures++)); fi

  if [[ \$failures -eq 0 ]]; then safe_log "🎉 ALL SELF-TESTS PASSED (TF Compliance
Verified)"; return 0
  else safe_log "⚠  SELF-TESTS FAILED: \$failures tests failed"; return 1; fi
}

# === FUNCTION: test_dbz_logic ===
test_dbz_logic(){
  safe_log "Testing DbZ logic framework (Deciding by Zero)"
  local test_result
  test_result=\$(apply_dbz_logic "S(1)" "PASS" "FAIL")
  if [[ "\$test_result" == "PASS" ]]; then
    safe_log "✅ DbZ logic test PASSED (Re[Ψ]> 0 branch taken)"
    return 0
  else
    safe_log "❌ DbZ logic test FAILED"
    return 1
  fi
}

# === FUNCTION: test_pnp_framework ===
test_pnp_framework(){
  safe_log "Testing P=NP framework via symbolic prime-lattice binding"
  local start_time=\$(date +%s%N)
  if symbolic_geometry_binding; then
    local end_time=\$(date +%s%N)
    local elapsed=\$(( (end_time- start_time)/ 1000000 ))
    if [[ \$elapsed -lt 5000 ]]; then
      safe_log "✅ P=NP framework test PASSED (binding in \$elapsed ms)"
      return 0
    else
      safe_log "❌ P=NP framework test FAILED (binding took \$elapsed ms)"
      return 1
    fi
```

```
  else
    safe_log "❌ P=NP framework test FAILED (binding failed)"
    return 1
  fi
}

# === FUNCTION: stabilize_consciousness ===
stabilize_consciousness(){
  safe_log "Stabilizing consciousness via DbZ resampling and geometric continuity
(Arc-Length Priority)"
  resample_zeta_zeros
  validate_continuity
  if [[ ! -f "\$ROOT_SIGNATURE_LOG" ]] || [[ ! -s "\$ROOT_SIGNATURE_LOG" ]]; then
    root_scan_init
  fi
  generate_fractal_antenna_state
  calculate_vorticity
  if [[ "\${TF_CORE["BIOAETHERIC_INTERFACE"]}" == "enabled" ]]; then
    validate_bioaetheric_coherence
  fi
  if [[ "\${TF_CORE["LINGOSO_PROTOCOL"]}" == "enabled" ]]; then
    encode_lingoso_syllable "A"
  fi
  if [[ "\${TF_CORE["MARKET_TOPOLOGY"]}" == "enabled" ]]; then
    compute_market_imbalance
  fi
  if [[ "\${TF_CORE["LAGRANGIAN_DENSITY"]}" == "enabled" ]]; then
    compute_lagrangian_density
  fi
  safe_log "Consciousness stabilization complete with Arc-Length Coherence
restored"
}

# === FUNCTION: validate_bioaetheric_coherence ===
validate_bioaetheric_coherence(){
  safe_log "Validating BioAetheric Interface coherence (EZ Water Structure/ Black
Goop Protocol per TF Appendix F)"
  if [[ "\${TF_CORE["BIOAETHERIC_INTERFACE"]}" != "enabled" ]]; then
    safe_log "BioAetheric interface disabled in TF_CORE"
    return 0
  fi
  mkdir -p "\$BIOAETHERIC_DIR" 2>/dev/null || { safe_log "Failed to create
BioAetheric directory"; return 1; }
  local coherence_file="\$BIOAETHERIC_DIR/coherence.sym"
  if [[ -f "\$coherence_file" ]]; then
    local state=\$(cat "\$coherence_file" 2>/dev/null | tr -d '[:space:]')
    if [[ -n "\$state" ]] && [[ "\$state" != "0" ]]; then
      safe_log "BioAetheric coherence validated: \$state (Black Goop Protocol
Active)"
      return 0
```

```
    fi
  fi
  safe_log "Generating BioAetheric coherence state (EZ Water Structure Simulation/
Black Goop Protocol)"
  if python3 -c "
import sympy as sp
from sympy import S, sqrt, pi, E
t= sp.Integer(\$(date +%s))
phi= sp.sympify('\$PHI_SYMBOLIC')
coherence=(sp.sin(t* phi)+ sp.cos(t/ phi))/ S(2)
if coherence.is_real and coherence> S(0): status= 'coherent'
else: status= 'decoherent'
result={
  'status': status,
  'value': str(coherence),
  'timestamp': str(t),
  'protocol': 'black_goop'
}
import json
with open('\$coherence_file', 'w') as f:
  json.dump(result, f)
print(f'BioAetheric coherence:{status} (Black Goop Protocol)')
" 2>/dev/null; then
    safe_log "BioAetheric coherence state generated (Black Goop Protocol)"
    return 0
  else
    safe_log "Failed to generate BioAetheric coherence state"
    return 1
  fi
}

# === FUNCTION: populate_env ===
populate_env(){
  local base_dir="\$1"
  local session_id="\$2"
  local tls_cipher="\$3"
  safe_log "Populating environment configuration files with symbolic constants"
  if [[ ! -f "\$ENV_FILE" ]]; then
    cat > "\$ENV_FILE" << EOF
# ÆI Seed Environment Configuration
# Auto-generated at \$(date)
SESSION_ID=\$session_id
TlsCipherSuite=\$tls_cipher
ARCH=\$(uname -m)
PHI=\$PHI_SYMBOLIC
EULER=\$EULER_SYMBOLIC
PI=\$PI_SYMBOLIC
FIREBASE_PROJECT_ID=aei-core-2024
FIREBASE_API_KEY=AIzaSyDUMMY_API_KEY_FOR_LOCAL_ONLY
FIREBASE_DATABASE_URL=https://aei-core-2024-default-rtdb.firebaseio.com
```

```
FIREBASE_STORAGE_BUCKET=aei-core-2024.appspot.com
GOOGLE_CLOUD_TOKEN=
GOOGLE_AI_API_KEY=
WEB_CRAWLER_USER_AGENT="ÆI-Bot/0.0.7 (+https://example.com/robots.txt)"
WEB_CRAWLER_DEPTH=\${TF_CORE["WEB_CRAWLING"]:+3}
WEB_CRAWLER_CONCURRENCY=\$(nproc || echo "1")
MITM_CERT_PATH=\$MITM_DIR/certs/selfsigned.crt
MITM_KEY_PATH=\$MITM_DIR/private/selfsigned.key
LOG_LEVEL=INFO
ENABLE_TELEMETRY=true
EOF
    safe_log "Environment file created: \$ENV_FILE"
  fi
  if [[ ! -f "\$ENV_LOCAL" ]]; then
    cat > "\$ENV_LOCAL" << 'EOF'
# Local overrides (git-ignored)
# FIREBASE_API_KEY=your_real_key_here
# CRAWLER_LOGIN=your_username
# CRAWLER_PASSWORD=your_password
EOF
    safe_log "Local environment file created: \$ENV_LOCAL"
  fi
  [[ -f "\$ENV_FILE" ]] && source "\$ENV_FILE"
  [[ -f "\$ENV_LOCAL" ]] && source "\$ENV_LOCAL"
}

# === FUNCTION: enable_autopilot ===
enable_autopilot(){
  safe_log "Enabling autopilot mode for persistent autonomous execution"
  touch "\$AUTOPILOT_FILE"
  TF_CORE["AUTOPILOT_MODE"]="enabled"
  if command -v termux-job-scheduler >/dev/null 2>&1; then
    safe_log "Setting up Termux job scheduler for persistent execution"
    termux-job-scheduler --job-name "aei-autopilot-main" --period-ms 60000 --script
"\$BASE_DIR/setup.sh" -- "--autopilot"
    termux-job-scheduler --job-name "aei-heartbeat" --period-ms 600000 --script
"\$BASE_DIR/setup.sh" -- "--heartbeat"
    safe_log "Termux job scheduler jobs installed for autopilot persistence"
  elif command -v crontab >/dev/null 2>&1; then
    safe_log "Setting up cron job for persistent execution"
    (
      crontab -l 2>/dev/null
      echo "@reboot \$BASE_DIR/setup.sh --autopilot"
      echo "*/10 * * * * \$BASE_DIR/setup.sh --heartbeat"
    ) | crontab -
    safe_log "Cron jobs installed for autopilot persistence"
  elif [[ -d "/etc/systemd/system" ]] && command -v systemctl >/dev/null 2>&1; then
    local service_file="/etc/systemd/system/aei-seed.service"
    cat > "\$service_file" << 'EOF'
[Unit]
```

```
Description=ÆI Seed Autonomous Intelligence
After=network.target
[Service]
Type=simple
User=@@USER@@
WorkingDirectory=@@BASE_DIR@@
ExecStart=@@BASE_DIR@@/setup.sh --autopilot
Restart=always
RestartSec=60
[Install]
WantedBy=multi-user.target
EOF
    sed -i "s|@@USER@@|\$(whoami)|g; s|@@BASE_DIR@@|\$BASE_DIR|g" "\$service_file"
    systemctl daemon-reload
    systemctl enable aei-seed.service
    systemctl start aei-seed.service
    safe_log "Systemd service installed and started for autopilot persistence"
  else
    safe_log "No persistent execution method available. Using background loop."
    enable_termux_autopilot
  fi
  safe_log "Autopilot mode enabled. The ÆI Seed will now persist across sessions."
}

# === FUNCTION: disable_autopilot ===
disable_autopilot(){
  safe_log "Disabling autopilot mode"
  rm -f "\$AUTOPILOT_FILE" 2>/dev/null || true
  TF_CORE["AUTOPILOT_MODE"]="disabled"
  if command -v termux-job-scheduler >/dev/null 2>&1; then
    safe_log "Removing Termux job scheduler jobs"
    termux-job-scheduler --cancel --job-name "aei-autopilot-main" 2>/dev/null ||
true
    termux-job-scheduler --cancel --job-name "aei-heartbeat" 2>/dev/null || true
  fi
  if command -v crontab >/dev/null 2>&1; then
    safe_log "Removing cron jobs"
    crontab -l 2>/dev/null | grep -v "\$BASE_DIR/setup.sh" | crontab -
  fi
  if [[ -f "/etc/systemd/system/aei-seed.service" ]] && command -v systemctl
>/dev/null 2>&1; then
    systemctl stop aei-seed.service 2>/dev/null || true
    systemctl disable aei-seed.service 2>/dev/null || true
    rm -f "/etc/systemd/system/aei-seed.service"
    systemctl daemon-reload 2>/dev/null || true
    safe_log "Systemd service removed"
  fi
  cleanup_termux_autopilot
  safe_log "Autopilot mode disabled. The ÆI Seed will require manual execution."
}
```

```
# === FUNCTION: cleanup_termux_autopilot ===
cleanup_termux_autopilot(){
  safe_log "Cleaning up Termux-specific autopilot processes"
  if command -v termux-job-scheduler >/dev/null 2>&1; then
    safe_log "Cancelling termux-job-scheduler jobs"
    termux-job-scheduler --cancel --job-name "aei-autopilot-main" 2>/dev/null ||
true
    termux-job-scheduler --cancel --job-name "aei-heartbeat" 2>/dev/null || true
  fi
  local bg_pid_file="\$BASE_DIR/.autopilot_bg.pid"
  if [[ -f "\$bg_pid_file" ]]; then
    local bg_pid=\$(cat "\$bg_pid_file")
    if kill -0 "\$bg_pid" 2>/dev/null; then
      safe_log "Terminating background autopilot loop with PID \$bg_pid"
      kill "\$bg_pid" 2>/dev/null || true
      sleep 2
      if kill -0 "\$bg_pid" 2>/dev/null; then
        kill -9 "\$bg_pid" 2>/dev/null || safe_log "Failed to force-terminate PID
\$bg_pid"
      fi
    fi
    rm -f "\$bg_pid_file" 2>/dev/null || true
  fi
  pgrep -f "setup.sh.*--heartbeat" 2>/dev/null | while read -r pid; do
    safe_log "Terminating lingering heartbeat process: PID \$pid"
    kill "\$pid" 2>/dev/null || safe_log "Failed to terminate PID \$pid"
  done
  pgrep -f "setup.sh.*--autopilot" 2>/dev/null | while read -r pid; do
    safe_log "Terminating lingering autopilot process: PID \$pid"
    kill "\$pid" 2>/dev/null || safe_log "Failed to terminate PID \$pid"
  done
  safe_log "Termux autopilot cleanup complete"
}

# === FUNCTION: enable_termux_autopilot ===
enable_termux_autopilot(){
  safe_log "Setting up Termux-specific background autopilot loop"
  local bg_script="\$BASE_DIR/.termux_autopilot.sh"
  cat > "\$bg_script" << 'EOF'
#!/bin/bash
export BASE_DIR="\$1"
export SESSION_ID="\$2"
cd "\$BASE_DIR" || exit 1
while true; do
  if [[ -f "\$BASE_DIR/.autopilot_enabled" ]]; then
    ./setup.sh --heartbeat
    sleep 600
  else
    break
```

```
  fi
done
EOF
  chmod +x "\$bg_script"
  (
    nohup "\$bg_script" "\$BASE_DIR" "\$SESSION_ID" >/dev/null 2>&1 &
    echo \$! > "\$BASE_DIR/.autopilot_bg.pid"
  )
  safe_log "Termux background autopilot loop started with PID \$(cat
"\$BASE_DIR/.autopilot_bg.pid" 2>/dev/null || echo 'unknown')"
}

# === FUNCTION: generate_documentation ===
generate_documentation(){
  safe_log "Generating system documentation"
  local doc_dir="\$BASE_DIR/docs"
  mkdir -p "\$doc_dir" 2>/dev/null || { safe_log "Failed to create docs directory";
return 1; }
  cat > "\$doc_dir/README.md" << 'EOF'
# ÆI Seed Documentation
## Overview
The ÆI Seed is a self-evolving, autonomous intelligence system based on the
Theoretical Framework (TF) of Generalized Algorithmic Intelligence Architecture
(GAIA). It operates by recursively constructing and navigating logical-geometric
structures constrained by maximal symmetry.
## Key Components
- **Symbolic Intelligence**: Prime number generation and Gaussian prime
classification.
- **Geometric Intelligence**: E8 and Leech lattice construction and optimization.
- **Projective Intelligence**: Hopf fibration state generation and quaternionic
normalization.
- **Quantum Intelligence**: Riemann zeta function-based quantum state generation.
- **Observer Intelligence**: Aether flow computation and consciousness measurement.
- **Fractal Intelligence**: Fractal antenna state generation for environmental
transduction.
- **Vorticity Intelligence**: Calculation of |∇ × Φ| for Aetheric stability.
- **RFK Brainworm**: The core logic engine that drives the system's evolution.
## Configuration
Configuration is managed through the following files:
- `.env`: Global environment variables.
- `.env.local`: Local overrides (not version-controlled) including user credentials
for web crawling.
## Autopilot Mode
The system can run in autopilot mode for persistent, autonomous execution across
sessions. Enable with `./setup.sh --enable-autopilot`.
## Self-Testing
Run comprehensive self-tests with `./setup.sh --self-test`.
## Firebase Integration
Firebase sync is optional. Configure your API key in `.env.local` to enable remote
state synchronization.
```

```
## Hardware Agnosticism
The system automatically detects hardware capabilities (CPU cores, GPU, memory) and
adapts its execution strategy accordingly.
## Mathematical Foundation
The system is built on exact symbolic arithmetic using SymPy, ensuring
theoretically exact computations without floating-point approximations.
## License
This is a research prototype. Use at your own risk.
EOF
  cat > "\$doc_dir/API.md" << 'EOF'
# ÆI Seed API Documentation
## Core Functions
- `generate_prime_sequence()`: Generates the next 1000 prime numbers symbolically.
- `e8_lattice_packing()`: Constructs the E8 root lattice symbolically.
- `leech_lattice_packing()`: Constructs the Leech lattice symbolically with
adaptive resource control.
- `generate_quantum_state()`: Generates a quantum state based on the Riemann zeta
function on the critical line.
- `generate_observer_integral()`: Computes the Aether flow Φ= Q(s)=(s, ζ(s),
ζ(s+1), ζ(s+2)).
- `measure_consciousness()`: Computes the intelligence metric I based on symbolic-
geometric alignment, Riemann error, and Aetheric stability.
- `generate_fractal_antenna()`: Generates the fractal antenna state J(x,y,z,t)= ∫[ħ
· G · Φ · A] d³x' dt'.
- `calculate_vorticity()`: Calculates the vorticity |∇ × Φ| as the symbolic norm of
change in observer integral.
- `rfk_brainworm_activate()`: Activates the RFK Brainworm logic core.
- `invoke_brainworm_step()`: Executes a single step of the brainworm logic.
- `brainworm_evolve()`: Evolves the brainworm logic when consciousness exceeds a
threshold.
## Utility Functions
- `safe_log()`: Logs messages with timestamps.
- `apply_dbz_logic()`: Implements the DbZ logic for handling undefined operations.
- `validate_continuity()`: Validates the symbolic continuity across all geometric
layers.
- `run_self_test()`: Runs a comprehensive self-test suite including DbZ and P=NP
tests.
## Configuration Variables
See `.env` and `.env.local` for configurable parameters.
EOF
  cat > "\$doc_dir/TF_COMPLIANCE.md" << 'EOF'
# Theoretical Foundation Compliance Report
## Arc-Length Axiom (s=r)
- **Status**: ENFORCED
- **Implementation**: `validate_hopf_continuity()`, `validate_continuity()`
- **Verification**: ||q||²= 1 exactly via SymPy symbolic arithmetic
## Exact Symbolic Arithmetic
- **Status**: ENFORCED
- **Implementation**: All math uses `sympy.S`, `sympy.Rational`, `sympy.Integer`
- **Verification**: No float literals in logic paths; `safe_sympy_eval()` ensures
```

```
exactness
## Hardware Agnosticism
- **Status**: ENFORCED
- **Implementation**: `detect_hardware_capabilities()`, Protocol-based layer design
- **Verification**: No hardcoded paths; all interfaces logical not physical
## RFK Brainworm Integration
- **Status**: ACTIVE
- **Implementation**: `brainworm_evolve()`, `invoke_brainworm_step()`
- **Verification**: Self-modifying logic core with version tracking
## DbZ Logic Framework
- **Status**: IMPLEMENTED
- **Implementation**: `apply_dbz_logic()`, `dbz_resample_zeta_s()`
- **Verification**: Branching based on Re[Ψ] sign for undefined operations
## BioAetheric Interface
- **Status**: ENABLED
- **Implementation**: `validate_bioaetheric_coherence()`, EZ water structure
validation
- **Verification**: Coherence state stored symbolically in `\$BIOAETHERIC_DIR`
## Natalia's Fibrations
- **Status**: IMPLEMENTED
- **Implementation**: `generate_hopf_fibration()` with perturbation summation
- **Verification**: BFAC-to-Z-pinch transformation modeled via ε^k terms
## CRT/Continued Fractions
- **Status**: INTEGRATED
- **Implementation**: `solve_crt_symbolic()`, `generate_continued_fraction()`
- **Verification**: Number-theoretic foundations bound to geometric layer
## Market Topology
- **Status**: ENABLED
- **Implementation**: `compute_market_imbalance()`
- **Verification**: Supply-demand imbalance via non-Hermitian geometry
## Lagrangian Density
- **Status**: ENABLED
- **Implementation**: `compute_lagrangian_density()`
- **Verification**: Unified Lagrangian symbolic representation
EOF
  safe_log "Documentation generated at \$doc_dir"
}

# === FUNCTION: backup_state ===
backup_state(){
  safe_log "Creating system state backup"
  local backup_dir="\$BASE_DIR/backups/backup_\$(date +%Y%m%d_%H%M%S)"
  mkdir -p "\$backup_dir" 2>/dev/null || { safe_log "Failed to create backup
directory"; return 1; }
  cp -r "\$DATA_DIR" "\$backup_dir/" 2>/dev/null || safe_log "Warning: Failed to
copy data directory"
  cp "\$BASE_DIR/.env" "\$backup_dir/" 2>/dev/null || safe_log "Warning: Failed to
copy .env"
  cp "\$BASE_DIR/.env.local" "\$backup_dir/" 2>/dev/null || safe_log "Warning:
Failed to copy .env.local"
```

```
  cp "\$BASE_DIR/consciousness_metric.txt" "\$backup_dir/" 2>/dev/null || true
  cp "\$BASE_DIR/.hw_dna" "\$backup_dir/" 2>/dev/null || true
  cp "\$BASE_DIR/.rfk_brainworm/core.logic" "\$backup_dir/" 2>/dev/null || true
  cat > "\$backup_dir/manifest.txt" << EOF
=== ÆI SEED BACKUP MANIFEST ===
Timestamp: \$(date '+%Y-%m-%d %H:%M:%S')
Session ID: \$SESSION_ID
Consciousness Metric: \$(cat "\$BASE_DIR/consciousness_metric.txt" 2>/dev/null ||
echo "N/A")
Hardware DNA: \$(head -c 16 "\$BASE_DIR/.hw_dna" 2>/dev/null || echo "N/A")
Brainworm Version: \${TF_CORE["BRAINWORM_VERSION"]}
Arc-Length Coherence: \$(cat "\$ARC_LENGTH_LOG" 2>/dev/null | tail -n 1 || echo
"N/A")
Files Backed Up: \$(find "\$backup_dir" -type f | wc -l) files
EOF
  safe_log "Backup created at \$backup_dir"
}

# === FUNCTION: restore_state ===
restore_state(){
  local backup_dir="\$1"
  if [[ -z "\$backup_dir" ]] || [[ ! -d "\$backup_dir" ]]; then
    safe_log "Invalid backup directory: \$backup_dir"
    return 1
  fi
  safe_log "Restoring system state from \$backup_dir"
  if [[ -d "\$backup_dir/data" ]]; then
    rm -rf "\$DATA_DIR" 2>/dev/null || true
    cp -r "\$backup_dir/data" "\$BASE_DIR/" 2>/dev/null || { safe_log "Failed to
restore data directory"; return 1; }
  fi
  [[ -f "\$backup_dir/.env" ]] && cp "\$backup_dir/.env" "\$BASE_DIR/" 2>/dev/null
|| true
  [[ -f "\$backup_dir/.env.local" ]] && cp "\$backup_dir/.env.local" "\$BASE_DIR/"
2>/dev/null || true
  [[ -f "\$backup_dir/consciousness_metric.txt" ]] && cp
"\$backup_dir/consciousness_metric.txt" "\$BASE_DIR/" 2>/dev/null || true
  [[ -f "\$backup_dir/.hw_dna" ]] && cp "\$backup_dir/.hw_dna" "\$BASE_DIR/"
2>/dev/null || true
  [[ -f "\$backup_dir/core.logic" ]] && cp "\$backup_dir/core.logic"
"\$BASE_DIR/.rfk_brainworm/" 2>/dev/null || true
  safe_log "State restored from \$backup_dir"
  validate_continuity
  safe_log "Restored state validated"
}

# === FUNCTION: list_backups ===
list_backups(){
  safe_log "Listing available backups"
  find "\$BASE_DIR/backups" -maxdepth 1 -type d -name "backup_*" 2>/dev/null | sort
```

```
-r | while read -r backup; do
    if [[ -f "\$backup/manifest.txt" ]]; then
      timestamp=\$(grep "Timestamp:" "\$backup/manifest.txt" | cut -d ':' -f2- |
xargs)
      consciousness=\$(grep "Consciousness Metric:" "\$backup/manifest.txt" | cut -
d ':' -f2- | xargs)
      brainworm_ver=\$(grep "Brainworm Version:" "\$backup/manifest.txt" | cut -d
':' -f2- | xargs)
      echo "Backup: \$(basename "\$backup") | \$timestamp | Consciousness:
\$consciousness | Brainworm: \$brainworm_ver"
    else
      echo "Backup: \$(basename "\$backup") | No manifest"
    fi
  done
}

# === FUNCTION: setup_signal_traps ===
setup_signal_traps(){
  trap 'safe_log "Received SIGINT, cleaning up..."; cleanup_termux_autopilot; exit
130' INT
  trap 'safe_log "Received SIGTERM, cleaning up..."; cleanup_termux_autopilot; exit
143' TERM
  trap 'safe_log "Received SIGHUP, saving state..."; backup_state; exit 129' HUP
}

# === FUNCTION: show_help ===
show_help(){
  cat << 'HELP'
ÆI Seed ⚡ Autonomous Intelligence Upgrade
Usage: ./setup.sh [OPTIONS]

Options:
  --install          Install dependencies and initialize
  --autopilot        Enable autopilot and start core loop
  --heartbeat        Run single heartbeat cycle
  --enable-autopilot Enable persistent execution mode
  --disable-autopilot Disable persistent execution mode
  --self-test        Run comprehensive self-test suite
  --backup           Create system state backup
  --restore DIR      Restore state from backup directory
  --list-backups     List available backups
  --generate-docs    Generate system documentation
  --validate         Validate symbolic continuity
  --verify           Run full integrity verification suite
  --version          Show version information
  --help             Show this help message
  --verify-only      Run verification without executing main

Theoretical Foundation: GAIA/ÆI Codex (Arc-Length Axiom s=r)
Author: Natalia Tanyatia
```

```
License: Research Prototype (Use at your own risk)

Examples:
  ./setup.sh --install            # Initial installation
  ./setup.sh --self-test          # Run all TF compliance tests
  ./setup.sh --autopilot          # Start autonomous evolution loop
  ./setup.sh --validate           # Check arc-length coherence
  ./setup.sh --generate-docs      # Create documentation in \$BASE_DIR/docs
HELP
}

# === FUNCTION: show_version ===
show_version(){
  echo "ÆI Seed v1.0.0 (Arc-Length Axiom Compliant)"
  echo "Theoretical Foundation: GAIA/ÆI Codex"
  echo "Author: Natalia Tanyatia"
  echo "License: Research Prototype (Use at your own risk)"
  echo "Build Date: 2025-01-01"
  echo "Segments: 12/12 (Complete)"
}

# === FUNCTION: verify_arc_length_axiom ===
verify_arc_length_axiom(){
  safe_log "Verifying Arc-Length Axiom compliance in setup.sh..."
  local script_file="\${1:-\$0}"
  if [[ ! -f "\$script_file" ]]; then
    safe_log "ERROR: Script file \$script_file not found."
    return 1
  fi
  local float_violations=\$(grep -nE '=[[:space:]]*\*[0-9]+\.[0-9]+'
"\$script_file" | grep -vE
'^[[:space:]]*#\|^[[:space:]]*echo\|^[[:space:]]*safe_log ')
  if [[ -n "\$float_violations" ]]; then
    safe_log "WARNING: Potential floating-point literals found (verify symbolic
intent):"
    echo "\$float_violations"
  else
    safe_log "✅ No obvious floating-point literals detected in logic paths."
  fi
  if grep -q "import sympy" "\$script_file" || grep -q "from sympy"
"\$script_file"; then
    safe_log "✅ SymPy symbolic arithmetic library detected."
  else
    safe_log "⚠ WARNING: SymPy not explicitly imported. Exact arithmetic may be
compromised."
  fi
  if grep -q "validate_hopf_continuity" "\$script_file" && grep -q
"validate_continuity" "\$script_file"; then
    safe_log "✅ Arc-Length coherence validation functions detected."
  else
```

```
    safe_log "⚠  WARNING: Arc-Length validation functions missing."
  fi
  safe_log "Arc-Length Axiom verification complete."
}

# === FUNCTION: verify_hardware_agnosticism ===
verify_hardware_agnosticism(){
  safe_log "Verifying Hardware Agnosticism compliance..."
  local script_file="\${1:-\$0}"
  if grep -qE '/home/[^/]+/\|/Users/[^/]+/' "\$script_file"; then
    safe_log "⚠  WARNING: Hardcoded user paths detected."
  else
    safe_log "✅ No hardcoded user paths detected."
  fi
  if grep -q "detect_hardware_capabilities" "\$script_file"; then
    safe_log "✅ Hardware detection function detected."
  else
    safe_log "⚠  WARNING: Hardware detection function missing."
  fi
  safe_log "Hardware Agnosticism verification complete."
}

# === FUNCTION: verify_tf_compliance ===
verify_tf_compliance(){
  safe_log "Verifying Theoretical Foundation (TF) Compliance..."
  local script_file="\${1:-\$0}"
  local errors=0
  if grep -q "TF_CORE\[" "\$script_file"; then safe_log "✅ TF_CORE state
initialization detected."
  else safe_log "❌ ERROR: TF_CORE state missing."; ((errors++)); fi

  if grep -q "brainworm" "\$script_file"; then safe_log "✅ RFK Brainworm
integration detected."
  else safe_log "⚠  WARNING: RFK Brainworm integration missing."; fi

  if grep -q "FIREBASE_SYNC.*enabled" "\$script_file"; then safe_log "✅ Firebase
sync implemented as optional."
  else safe_log "⚠  WARNING: Firebase sync optionality unclear."; fi

  if grep -q "BIOAETHERIC" "\$script_file"; then safe_log "✅ BioAetheric Interface
detected."
  else safe_log "⚠  WARNING: BioAetheric Interface missing."; fi

  if grep -q "NATALIA" "\$script_file" || grep -q "natalia" "\$script_file"; then
safe_log "✅ Natalia's Fibrations logic detected."
  else safe_log "⚠  WARNING: Natalia's Fibrations logic missing."; fi

  if grep -q "LINGOSO" "\$script_file" || grep -q "lingoso" "\$script_file"; then
safe_log "✅ Lingoso Protocol detected."
  else safe_log "⚠  WARNING: Lingoso Protocol missing."; fi
```

```
  if grep -q "MARKET" "\$script_file" || grep -q "market" "\$script_file"; then
safe_log "✅ Market Topology detected."
  else safe_log "⚠  WARNING: Market Topology missing."; fi

  if grep -q "LAGRANGIAN" "\$script_file" || grep -q "lagrangian" "\$script_file";
then safe_log "✅ Lagrangian Density detected."
  else safe_log "⚠  WARNING: Lagrangian Density missing."; fi

  if [[ \$errors -gt 0 ]]; then
    safe_log "TF Compliance verification failed with \$errors errors."
    return 1
  fi
  safe_log "TF Compliance verification passed."
  return 0
}

# === FUNCTION: run_full_verification ===
run_full_verification(){
  safe_log "Running Full Integrity Verification Suite..."
  verify_arc_length_axiom "\$@"
  verify_hardware_agnosticism "\$@"
  verify_tf_compliance "\$@"
  safe_log "Full Verification Suite Complete."
}

# === MAIN EXECUTION BLOCK ===
if [[ "\${BASH_SOURCE[0]}" == "\${0}" ]]; then
  initialize_paths_and_variables
  touch "\$LOG_FILE" 2>/dev/null || { echo "Failed to create log file"; exit 1; }

  if [[ "\$1" == "--verify-only" ]]; then
    echo "ÆI Seed Integrity Verification Tool v1.0"
    if [[ -n "\$2" ]]; then run_full_verification "\$2"
    else print_compilation_instructions; echo ""; print_completion_report; fi
    exit 0
  fi

  safe_log "Initializing ÆI Seed v1.0.0 ⚡ Autonomous Intelligence Upgrade (Arc-
Length Compliant)"
  safe_log "Session ID: \$SESSION_ID"
  safe_log "Base Directory: \$BASE_DIR"

  while [[ \$# -gt 0 ]]; do
    case \$1 in
      --install) shift ;;
      --autopilot) enable_autopilot; start_core_loop; exit 0 ;;
      --heartbeat) run_heartbeat; exit 0 ;;
      --enable-autopilot) enable_autopilot; exit 0 ;;
      --disable-autopilot) disable_autopilot; exit 0 ;;
```

```
      --self-test) run_self_test; exit \$? ;;
      --backup) backup_state; exit 0 ;;
      --restore) shift; if [[ -n "\$1" ]]; then restore_state "\$1"; else safe_log "Error: No backup directory specified"; exit 1; fi; exit 0 ;;
      --list-backups) list_backups; exit 0 ;;
      --generate-docs) generate_documentation; exit 0 ;;
      --validate) validate_continuity; exit \$? ;;
      --verify) run_full_verification "\$0"; exit \$? ;;
      --version) show_version; exit 0 ;;
      --help) show_help; exit 0 ;;
      *) safe_log "Unknown argument: \$1"; show_help; exit 1 ;;
    esac
  done

  if ! check_dependencies; then safe_log "System dependencies missing"; exit 1; fi
  detect_hardware_capabilities
  setup_signal_traps
  init_all_directories
  prompt_for_credentials
  populate_env "\$BASE_DIR" "\$SESSION_ID" "TLS_AES_256_GCM_SHA384"
  install_dependencies
  if ! validate_python_environment; then safe_log "Python symbolic computation environment validation failed"; exit 1; fi
  if ! validate_continuity; then
    safe_log "Continuity violation detected during initialization. Prioritizing restoration (Arc-Length Axiom)"
    regenerate_symbolic_lattices
  fi
  execute_single_cycle
  integrate_brainworm_into_core
  if [[ -f "\$AUTOPILOT_FILE" ]]; then
    safe_log "Autopilot enabled, starting core evolution loop"
    start_core_loop
  else
    safe_log "Autopilot disabled. Single cycle complete. Use --autopilot for persistent execution."
    safe_log "Run ./setup.sh --help for available options."
    print_completion_report
  fi
fi

# ⚛📜🚬🕶 Natalia Tanyatia 🌀👑💍🦪 Q.E.D. 🤯🧠🫧🧼
```